

Database Systems (DVML3,DAT5,SW5) Operator Execution

Yumeng Song and Juan Manuel Rodriguez

Department of computer science, Aalborg University

Fall 2025



AALBORG
UNIVERSITY



DBMS Internals

- ▶ We know how to use a DBMS
- ▶ We know how to store and access data efficiently in a DBMS.
- ▶ We will look into **how to execute queries efficiently**.

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager

Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1-12.4, 13.2-13.4, 14.1-14.6, 15.4-15.6

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

- <https://15445.courses.cs.cmu.edu/fall2022/>



SQL Core Operators

→ Mapping to Relational Algebra

	Relational algebra	SQL
π	projection	SELECT DISTINCT
σ	selection	WHERE without nesting
$\bowtie$	join	FROM, WHERE FROM with JOIN or NATURAL JOIN
ρ	renaming	FROM with tuple variable; AS
$-$	difference	WHERE with nesting / EXCEPT
$\cap$	intersection	WHERE with nesting / INTERSECT
$\cup$	union	UNION
γ	grouping	GROUP BY



Query Plan & Query Execution

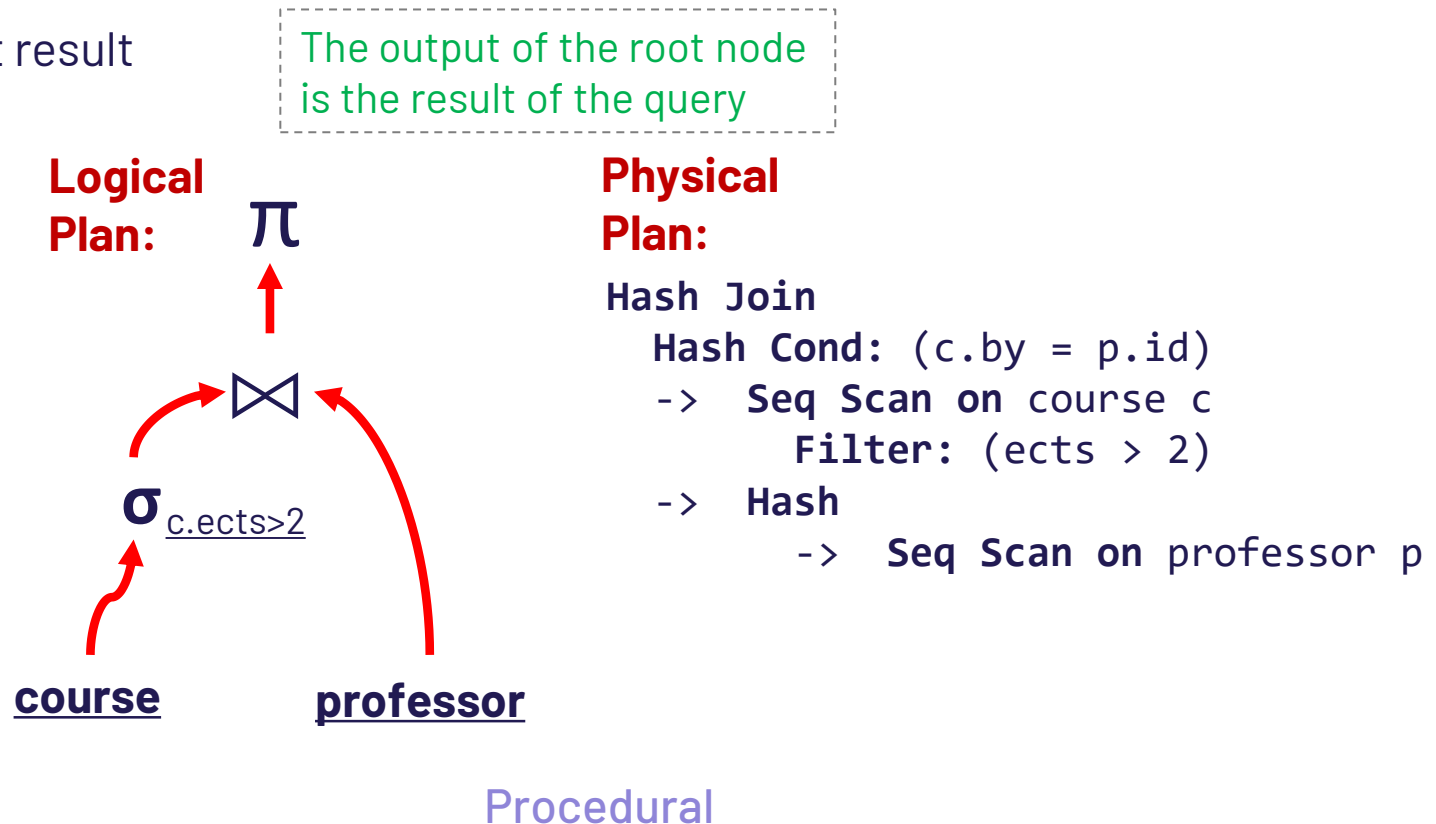
SQL queries are declarative

The DBMS needs to translate them into a series of operations to compute the correct result

Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Declarative



Disk-Oriented DBMS

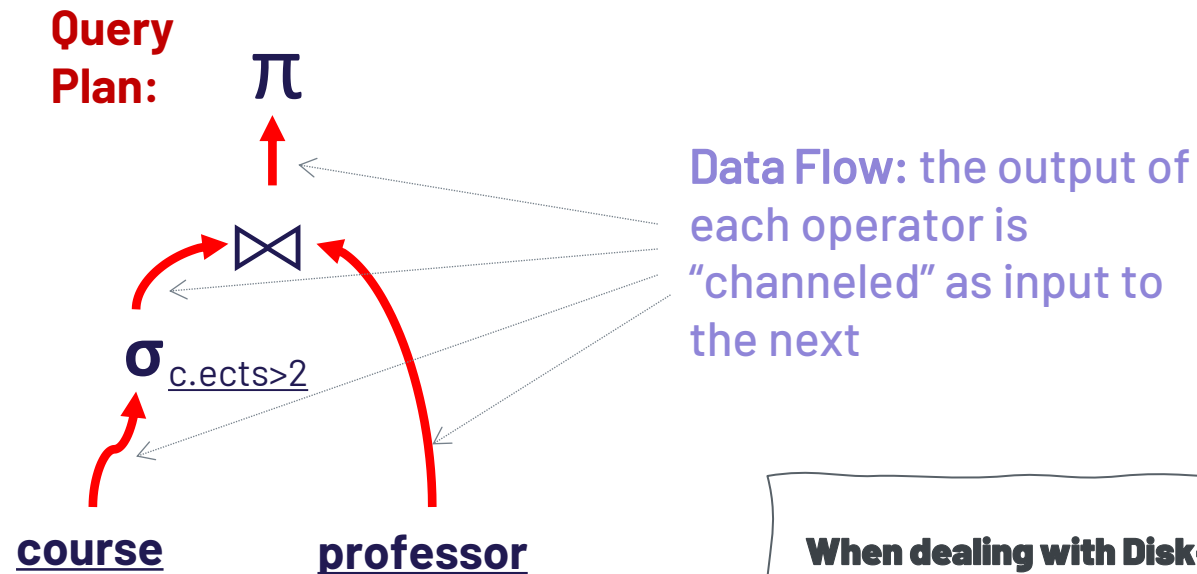
For simplicity, *in this lecture*, we use the Logical Plan figure and refer to physical implementation on top of it. We just call it **Query Plan**

Disk-Oriented: the data does not fit into main memory all the time!

We assume an entire **table may not fit** into main memory!

We **also** assume the **output of an operator may not fit** entirely into main memory!

Solution: compute partial results in memory, then "spill to disk" partial computations



When dealing with Disk-Based algorithms
We always prefer solutions that maximize the amount of sequential I/O



When do we need to sort?

The Relational model & SQL are unsorted!

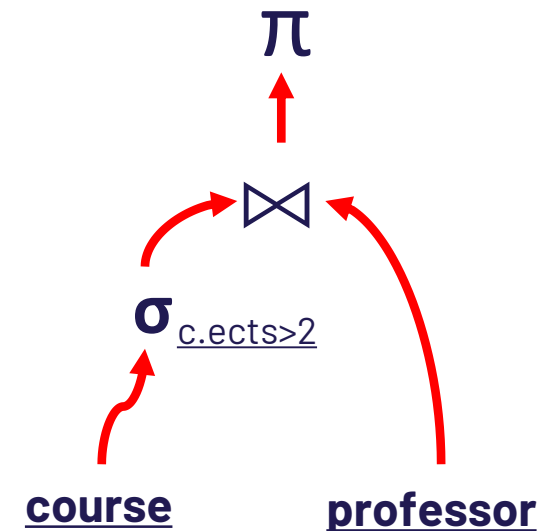
(Explicit requirement) **Queries may require** the output to be sorted (ORDER BY)!

(Implicit requirement) Sorting is used to **implement more efficiently** some operations:

- Duplicate elimination (DISTINCT)
- Bulk-load a B+Tree
- Compute aggregations
- **Some join implementations**

There are cases where the DBMS decides to sort the output of an operator without the user asking for it!

If we are accessing tuples that have been indexed, if the pages are clustered on the sorting key then we don't need to sort



External (Disk Based) Sorting

If the data to be sorted **fits in memory**, then we use **normal in memory sort** (e.g., quick sort)

If the data to be sorted **does not fit in memory**, **external sorting algorithms** are aware of the cost to read and save partial results to disk

Basic Strategy (divide and conquer)

1. Split data (pages) into small subsets (called "runs") → write to disk
2. Sort each individual run in isolation → write to disk
3. Given small sorted runs, merge them into a larger sorted runs (*repeat this step*) → write to disk after each step

Merge Sort

6 5 3 1 8 7 2 4

Basic Property:

if two collections are sorted internally,
generating a new sorted collection resulting
from the union of the **takes linear time** and
does **not require random access**.

see GIF on https://en.wikipedia.org/wiki/Merge_sort



Simple case: 2-way External Merge Sort (2EMS)

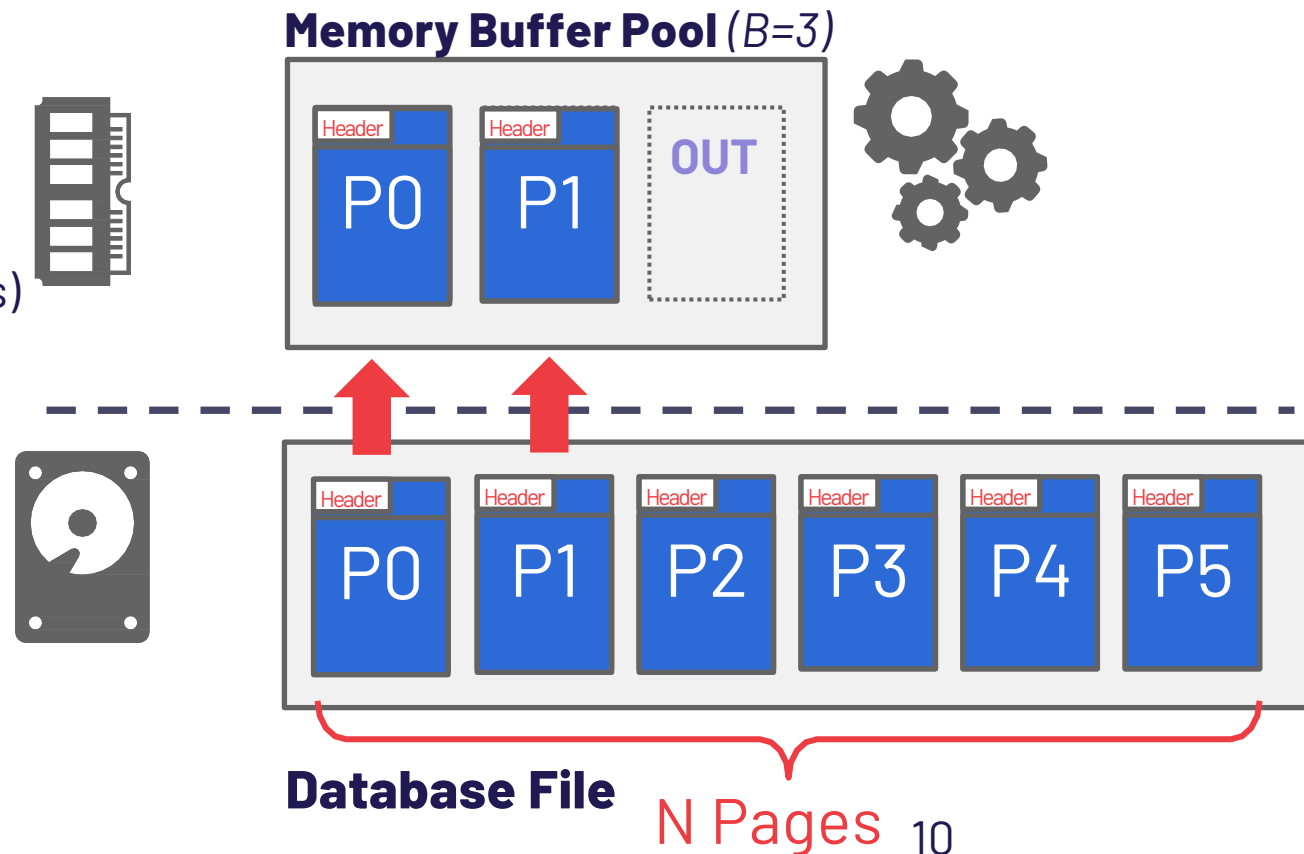
2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

K-way → we read at each step K runs and generate 1 sorted run that is the sum of all of them

Basic assumption:

- data is partitioned in **N pages** in total on disk
- **memory can fit only $B < N$ pages** (called B buffers)

We cannot use all B buffers, **we need to leave 1 out for writing to disk!**



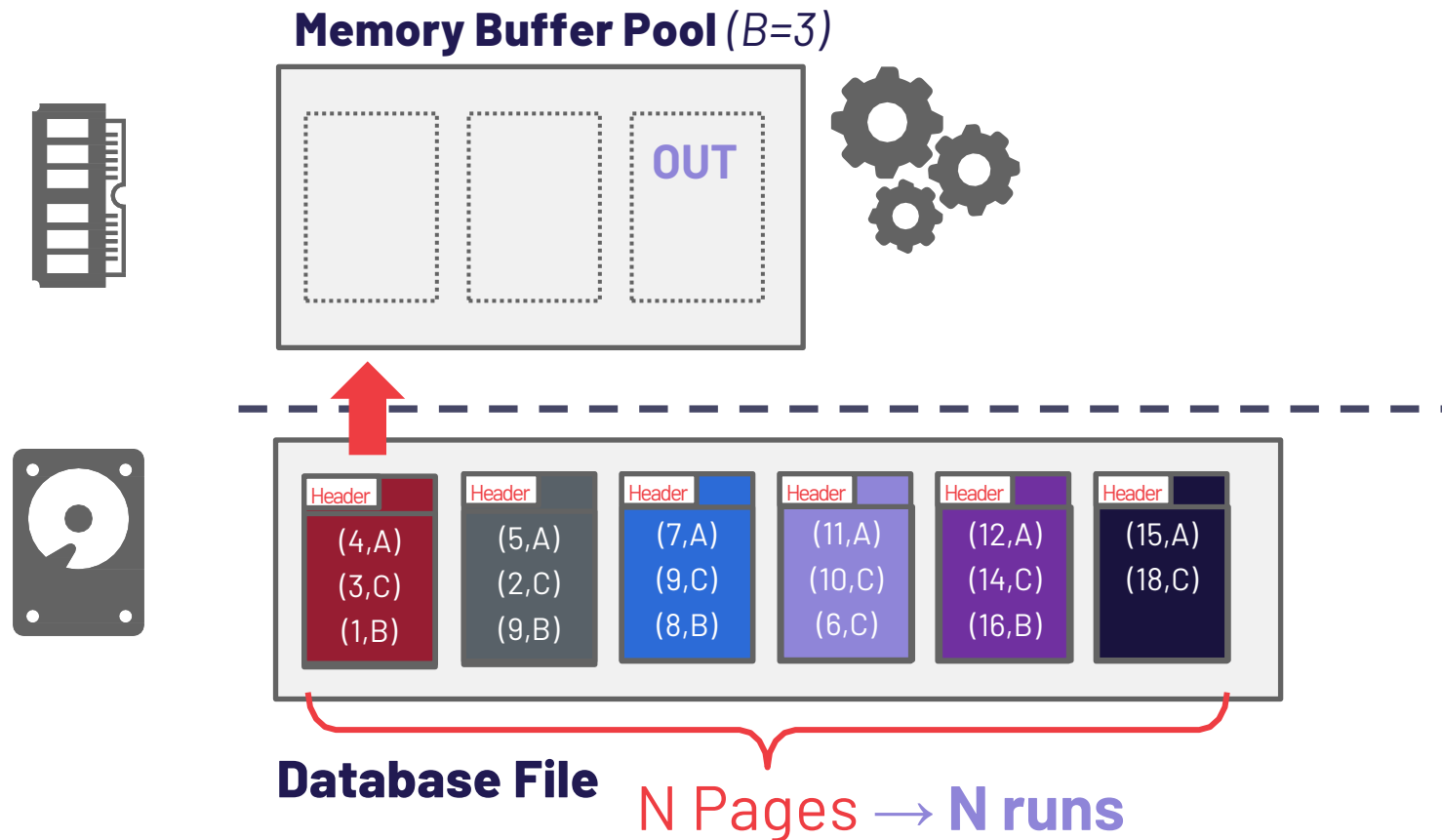


2EMS: Step-0

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Step-0: Before that we need to sort each single page

Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



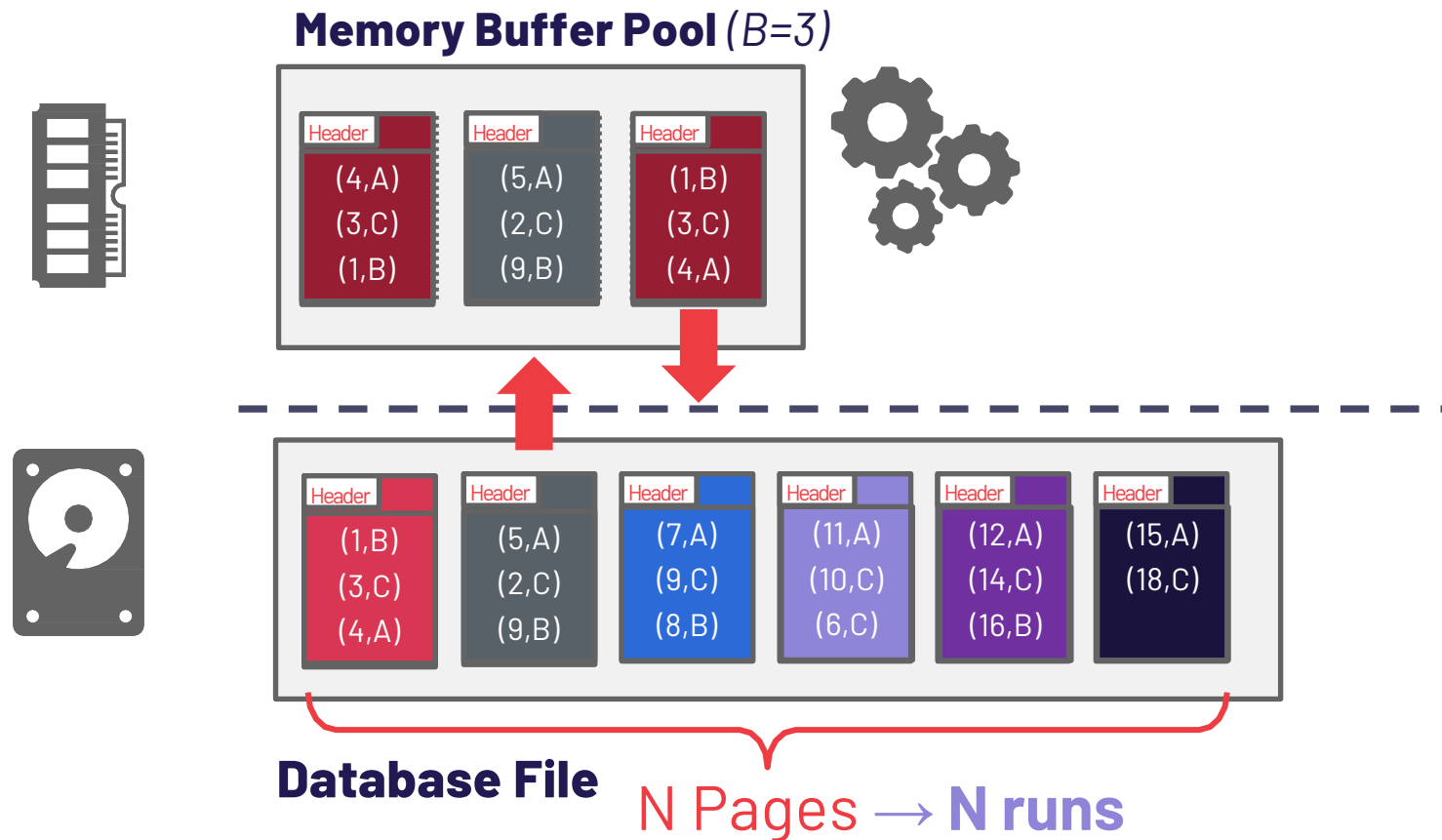


2EMS: Step-0

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Step-0: Before that we need to sort each single page

Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



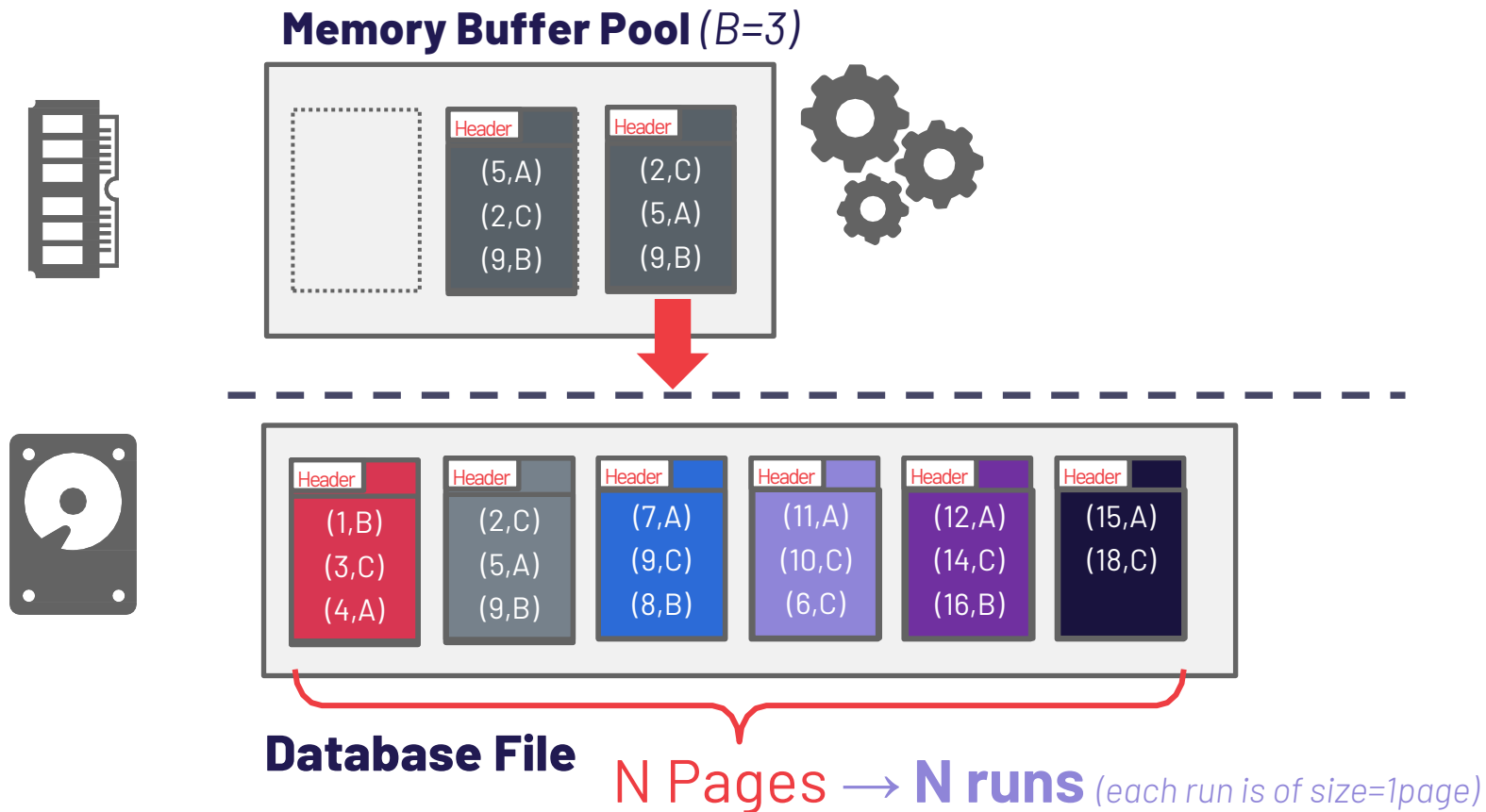


2EMS: Step-0

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Step-0: Before that we need to sort each single page

Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



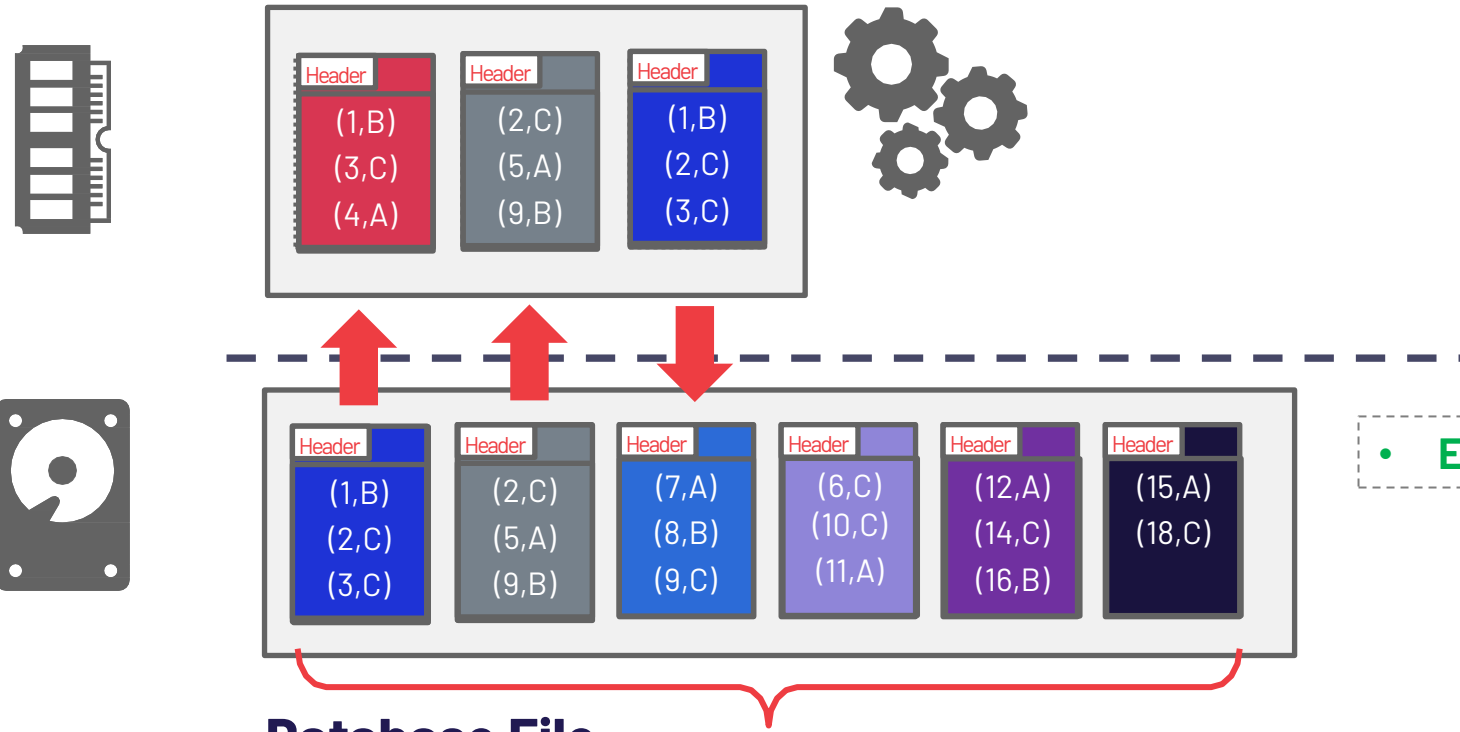


2EMS: Step-1

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Read 2 runs of size 1 page → produce 1 run of size 2 pages

Memory Buffer Pool ($B=3$)



Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only

Database File

N Pages → **N runs** (each run is of size=1page)

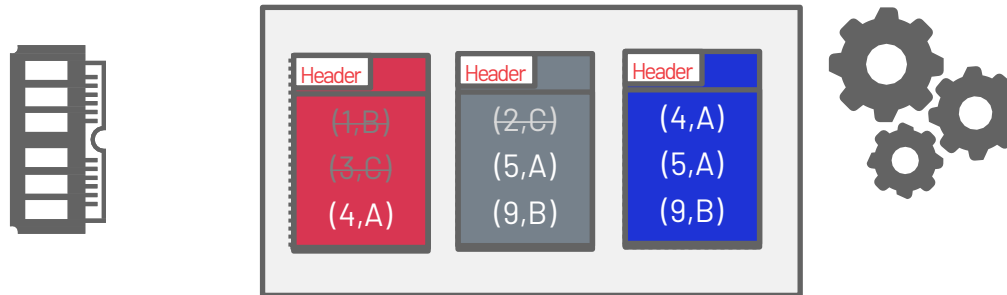


2EMS: Step-1

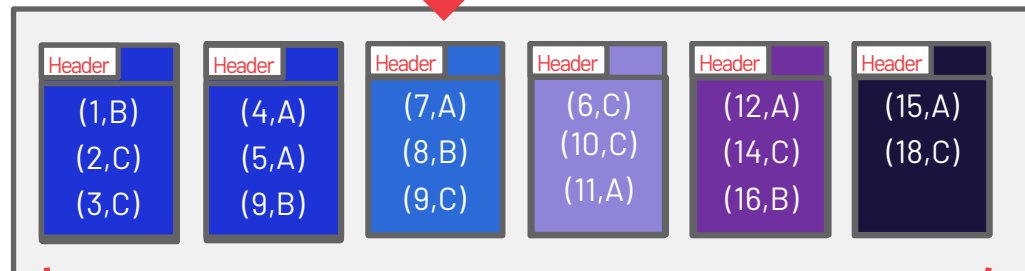
2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Read 2 runs of size 1 page → produce 1 run of size 2 pages

Memory Buffer Pool ($B=3$)



Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



- Each single page is sorted!

Database File

N Pages → **N runs** (each run is of size=1page)



2EMS: Step-2

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Read 2 runs of size 2 pages → **produce 1 run of size 4 pages!**

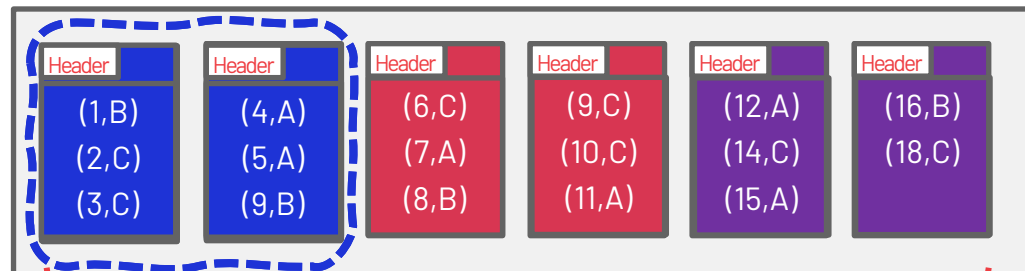
Memory Buffer Pool ($B=3$)



Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



One Run is
internally sorted!



- Each single page is sorted!
- Pairs of consecutive pages are sorted!

Database File

N Pages → **N/2 runs** (each run is of size=2pages)

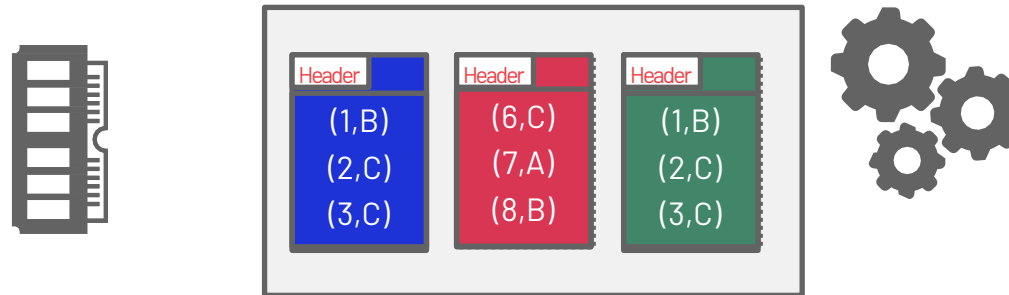


2EMS: Step-2

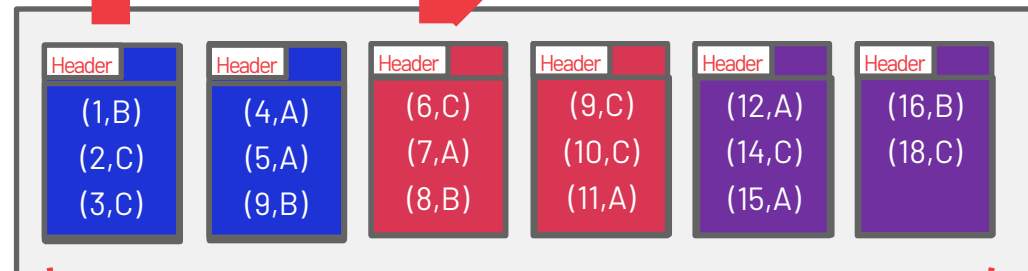
2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

Read 2 runs of size 2 pages → **produce 1 run of size 4 pages!**

Memory Buffer Pool ($B=3$)



Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



- Each single page is sorted!
- Pairs of consecutive pages are sorted!

Database File

N Pages → **N/2 runs** (each run is of size=2pages)

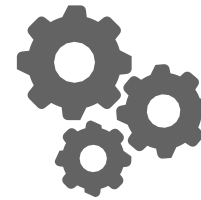
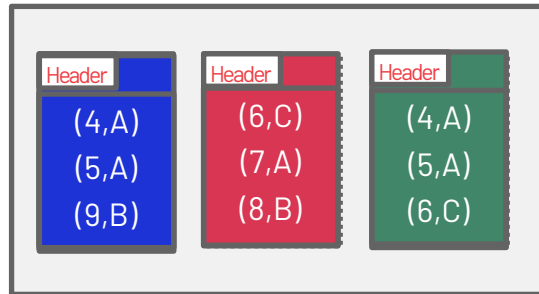


2EMS: Step-2

2-way → we read at each step 2 runs and generate 1 sorted run that is the sum of the two

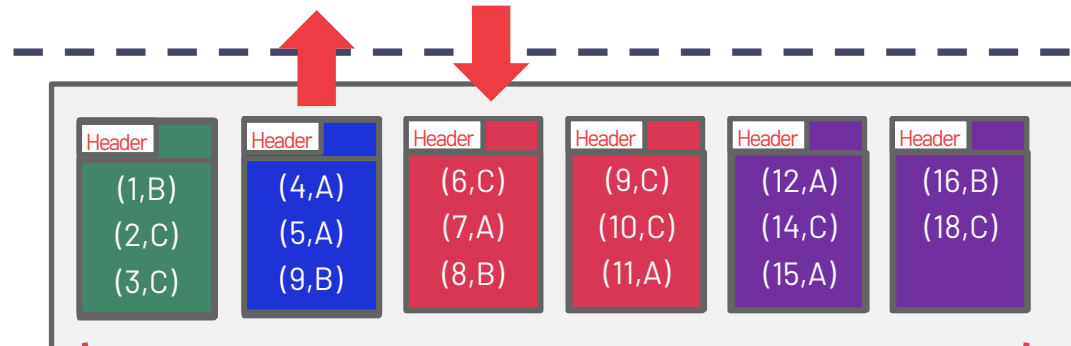
Read 2 runs of size 2 pages → **produce 1 run of size 4 pages!**

Memory Buffer Pool ($B=3$)



When we used all tuples from one page,
We keep processing pages from the runs
in order

Assume each tuple is:
(Key, SomeValue)
And we want to sort
according to Key only



- Each single page is sorted!
- Pairs of consecutive pages are sorted!

Database File

N Pages → **N/2 runs** (each run is of size=2pages)



2EMS: full process

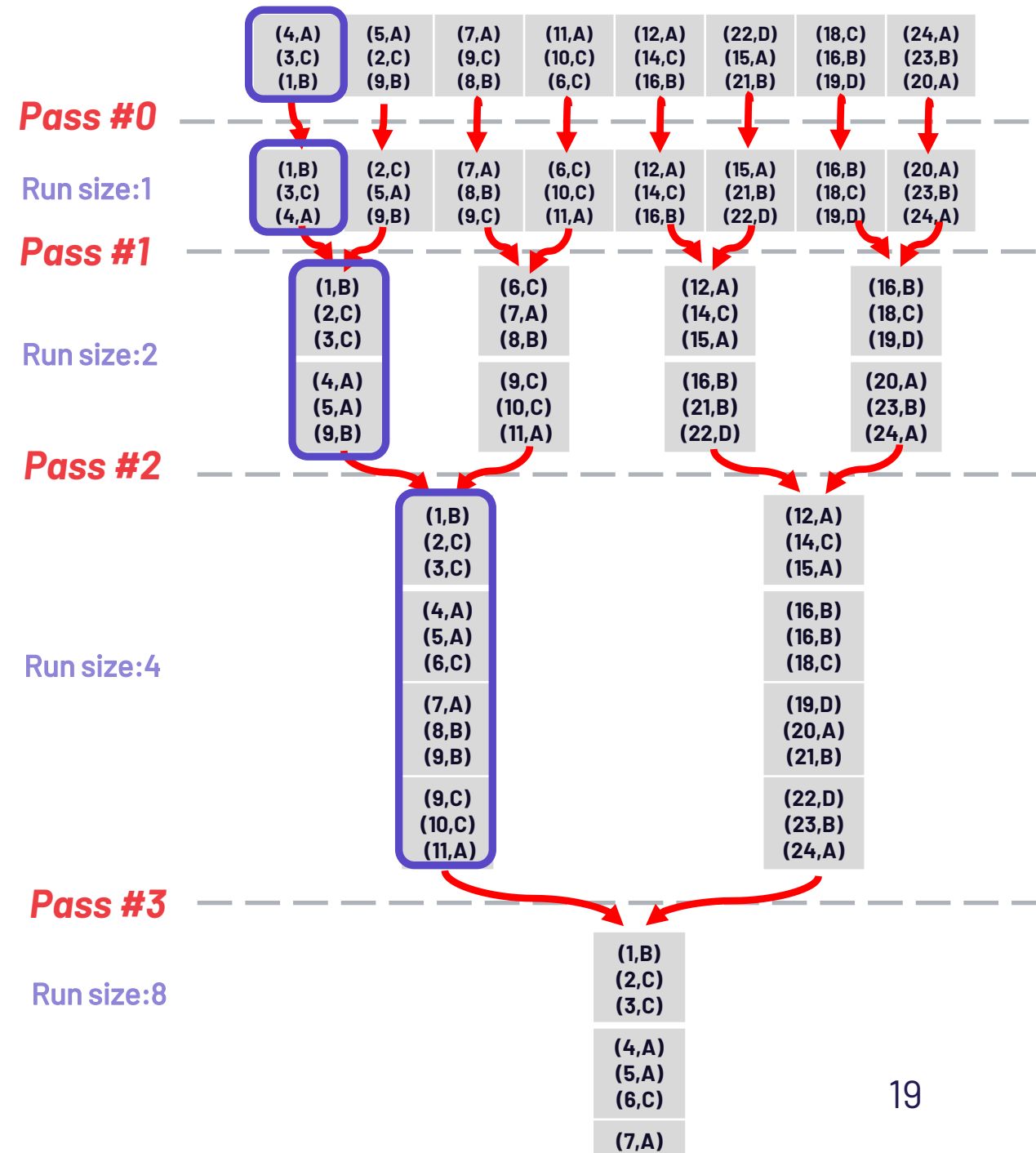
- 3 buffers, 2 used for merge-sort
- Number of passes = $1 + \lceil \log_2 N \rceil$
- Number of I/O = $2N \times (\text{\# of passes})$

Given 8 pages

- # Passes = $1 + \lceil \log_2(8) \rceil = 4$
- # I/O = $2 \times 8 \times 4 = 64$

Given 800 pages ($\times 4\text{Kb}$ page = 3.2MB)

- # Passes = $1 + \lceil \log_2(800) \rceil = 11$
- # I/O = $2 \times 800 \times 11 = 17600$
Read $\sim 35\text{MB}$ + Write $\sim 35\text{MB}$





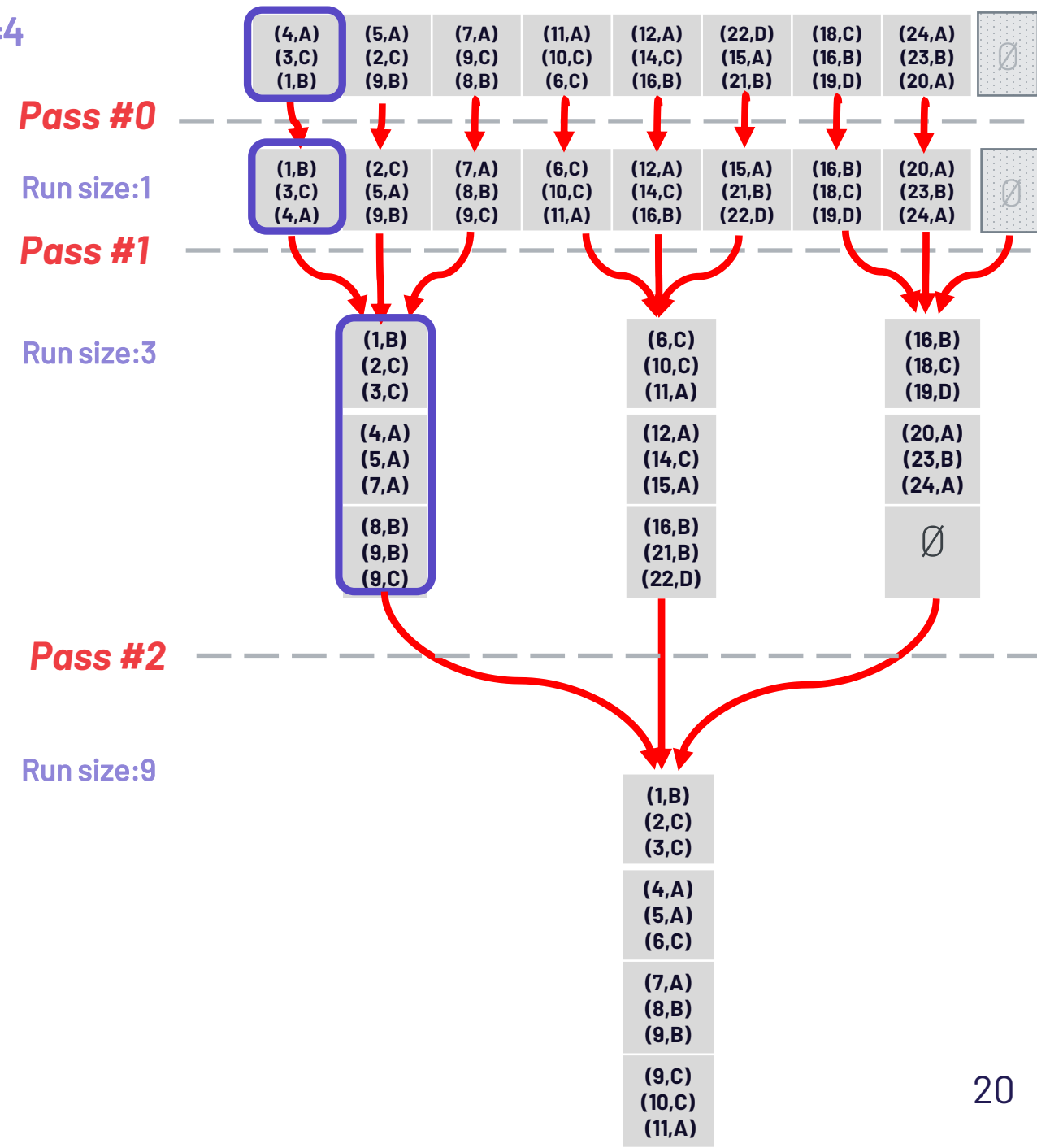
K-Way EMS

- **N** pages & **B** buffers, **K=B-1** used for merge-sort
- Initial number of Runs $\lceil N/K \rceil$
- Number of passes = $1 + \lceil \log_K(\lceil N/K \rceil) \rceil$
- Number of I/O = $2N \times (\text{\# of passes})$

Given $B=4$, $K=4-1=3$, 800 pages ($\times 4\text{Kb page} = 3.2\text{MB}$)

- **# Passes** = $1 + \lceil \log_3(800) \rceil = 8$
- **# I/O** = $2 \times 800 \times 8 = 12800$
Read $\sim 25\text{MB}$ + Write $\sim 25\text{MB}$

Assume $B=4$
then $K=3$



Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory ★

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1–13.3, 14.1–14.3, 14.5

– from *Database System Concepts* 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

– <https://15445.courses.cs.cmu.edu/fall2022/>



Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory ★

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

– from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

– <https://15445.courses.cs.cmu.edu/fall2022/>



Aggregations with Sorting

1. Find tuples that share the same value for the grouping attribute:

- Avoid iterating over all tuples too many times

2. Sorting: put tuples with the same value in contiguous places

- Grouping is then calculated on a single scan! (After the cost of sorting)
- We append to a temporary table and we update at most the last tuple

Can be used for DISTINCT as well!

```
SELECT c.Name, SUM(c.students)
FROM course c
GROUP BY c.Name;
```

Name	Rank	Students
Bob	C4	50
Bob	C8	60
Jane	C2	100
Zack	C2	120
Zack	C4	20
Zack	C3	100

Yet, there is no requirement for the output of DISTINCT/GROUP BY to be sorted!



Aggregations with Hashing

1. Find tuples that share the same value for the grouping attribute:

- Avoid iterating over all tuples too many times

2. Hashing: build a temporary hash table expecting redundant keys

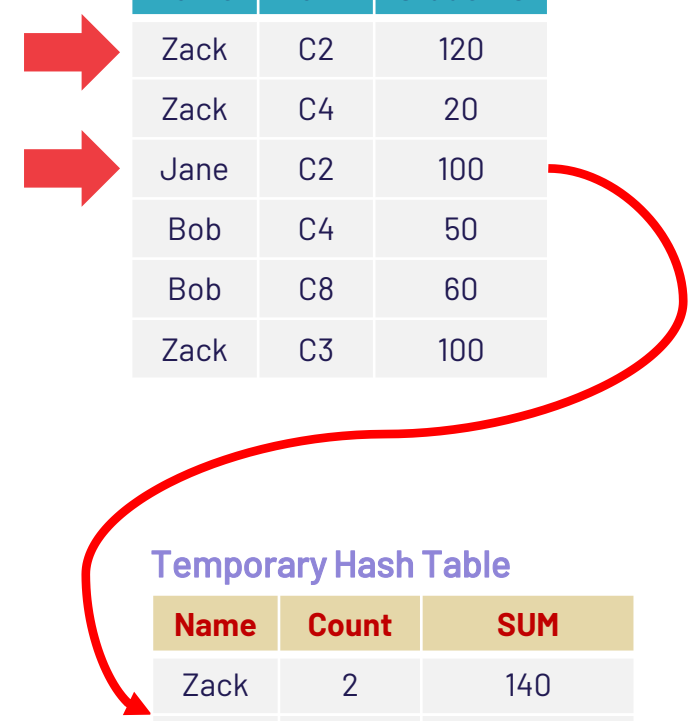
- Scan the data 1 time only.
- Given a record, find it's hashing slot, find if a value exists there already
- Insert or Update

For DISTINCT we discard records if we find a duplicate during hashing

For GROUP BY we store as value the temporary result of the aggregate

If the hash table do not fit into memory, then there exist approaches similar to external merge sort.

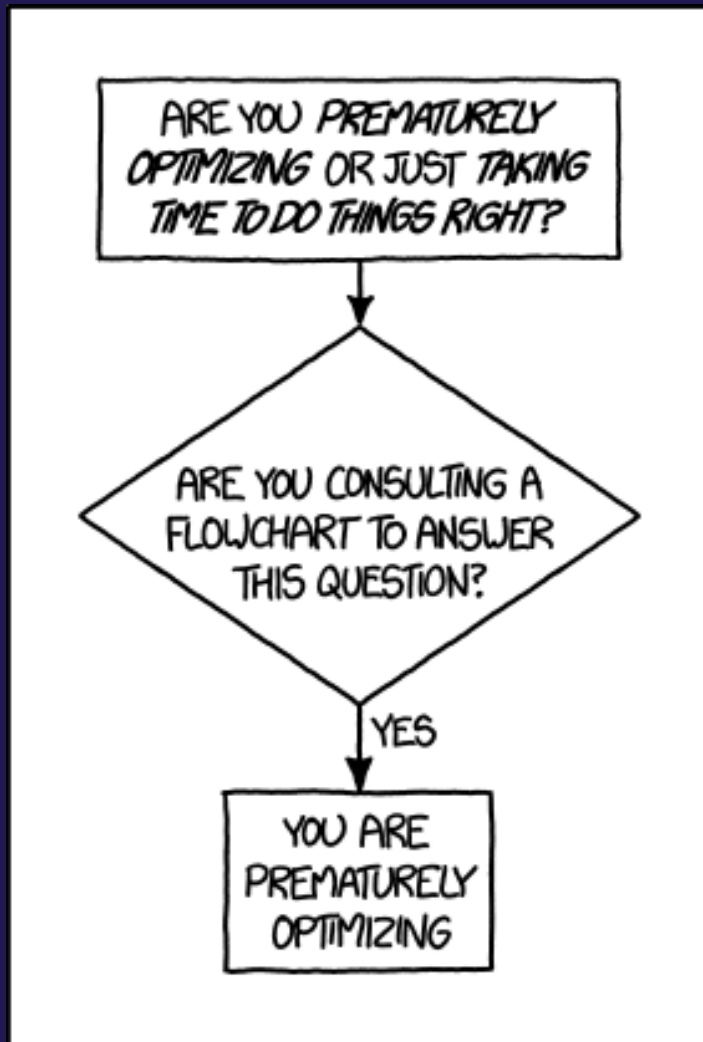
```
SELECT c.Name, AVG(c.students)
FROM course c
GROUP BY c.Name;
```



Name	Rank	Students
Zack	C2	120
Zack	C4	20
Jane	C2	100
Bob	C4	50
Bob	C8	60
Zack	C3	100

Temporary Hash Table

Name	Count	SUM
Zack	2	140
Jane	1	100



<https://m.xkcd.com/1691/>

Premature optimization is the root of all evil, so to start this project I'd better come up with a system that can determine whether a possible optimization is premature or not.



<https://m.xkcd.com/1445/>

I need an extension for my research project because I spent all month trying to figure out whether learning Dvorak would help me type it faster.

Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory ★

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1-13.3, 14.1-14.3, 14.5

– from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

– <https://15445.courses.cs.cmu.edu/fall2022/>



Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory ★

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1–13.3, 14.1–14.3, 14.5

– from *Database System Concepts* 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

– <https://15445.courses.cs.cmu.edu/fall2022/>





The Importance of Joins

Normalization splits information to avoid redundancies

Joins are needed to obtain information partitioned in multiple relations
This happens mainly through **Foreign Keys** → **use of binary equi-joins!**

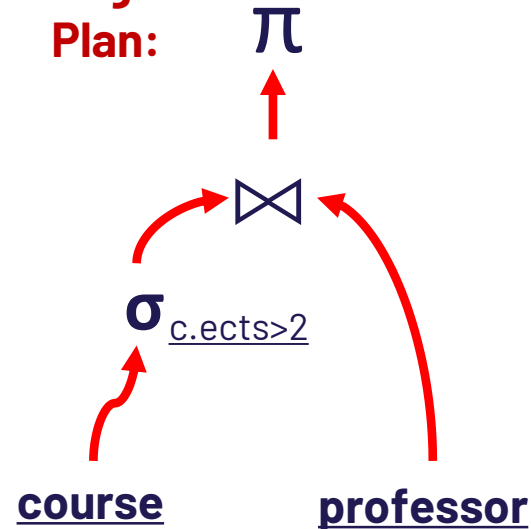
Query:

```
SELECT p.name, c.title, c.ects  
FROM course c JOIN professor p  
      ON p.id = c.taughtby  
WHERE c.ects > 2;
```

There are also multi-way joins

Declarative

Logical Plan:



Physical Plan:

Hash Join

Hash Cond: (c.by = p.id)
-> Seq Scan on course c
 Filter: (ects > 2)

-> Hash
 -> Seq Scan on professor p

We need an implementation that minimize I/O costs

Procedural

Focus

1. **Binary inner equi join** typical when we join on foreign keys
2. **M-way joins exist**, multiple tables on the same key (not much implemented, useful for data in the graph-model)
3. **Strategy: “smaller table out”** reduce cost by iterating first on the pages/tuples of the small table
4. **What algorithm to pick?** *What’s the cost?*
Based on what we need to produce (sorted? Batched?) and what we need to join

id	name	rank	salary
2125	Socrates	C4	100
2126	Russel	C4	100
2134	Augustinus	C3	200
2137	Kant	C4	200
...	...	...	...

id	title	ects	students	by
1	Basic	4	20	2137
2	Ethics	4	10	2125
3	Theory Cogn	3	50	2126
4	CS 101	2	40	2134
5	Math	8	20	2137
6	Morals	3	20	2140
...	...	...	...	...

Remember: Tables are stored in pages and even if we need few tuples from each page, we need to read the entire page!

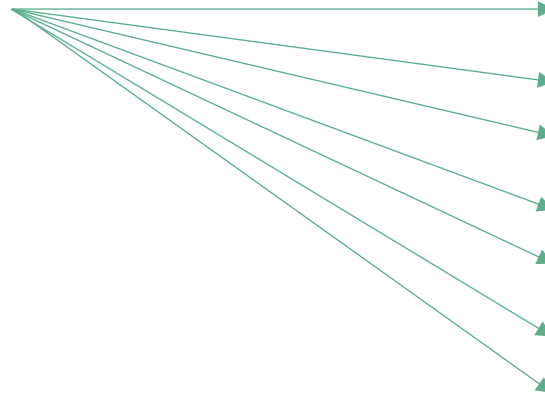
The Wrong Way

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

We never cross product and then filter!

We produce too many intermediate results that we throw away!

id	name	rank	salary
2125	Socrates	C4	100
2126	Russel	C4	100
2134	Augustinus	C3	200
2137	Kant	C4	200
...	...	...	...



id	title	ects	students	by
1	Basic	4	20	2137
2	Ethics	4	10	2125
3	Theory Cogn	3	50	2126
4	CS 101	2	40	2134
5	Math	8	20	2137
6	Morals	3	20	2140
...	...	...	...	...

```
SELECT p.id, p.name, ....  
FROM professor p, course c  
WHERE p.id = c.by AND ...
```

The DBMS will try to recognize when it is possible to use an equi-join even if we don't write it explicitly in the "FROM" clause



Naïve Nested Loop

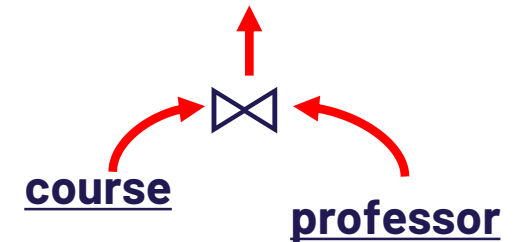
```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

```
foreach block  $B^P$  in P:  
  foreach tuple  $p$  in  $B^P$ :  
    foreach block  $B^C$  in C:  
      foreach tuple  $c$  in  $B^C$ :  
        if joinable(p,c):  
          emit (p,c)
```

Outer Table

Inner Table



The logical plan does not tell us which one should be outer and which should be inner

this will require $|P| \times |C|$ iterations.... but this is not the cost:

COST: For every tuple in P scan C
I/O cost: $N^P + (|P| \times N^C)$

This algorithm reads all the pages for every single tuple!



Naïve Nested Loop (Cost)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

$|P| = 10.000, N^P = 150$
 $|C| = 500.000, N^C = 500$

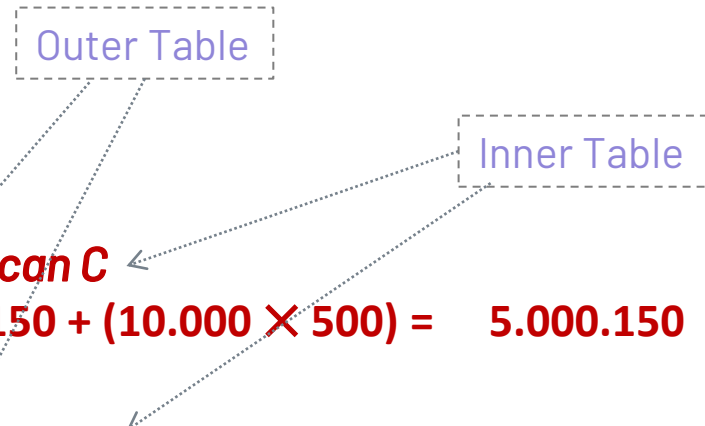
COST: For every tuple in P scan C
I/O cost: $N^P + (|P| \times N^C) = 150 + (10.000 \times 500) = 5.000.150$

COST: For every tuple in C scan P
I/O cost: $N^C + (|C| \times N^P) = 500 + (500.000 \times 150) = 75.000.500$

The logical plan does not tell us which one should be outer and which should be inner



We should always have the smaller table as the outer table!





Naïve Nested Loop (Cost - II)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

$|P| = 50.000$, $N^P = 250$
 $|C| = 20.000$, $N^C = 300$

COST: For every tuple in P scan C
I/O cost: $N^P + (|P| \times N^C) = 250 + (50.000 \times 300) = 15.000.250$

COST: For every tuple in C scan P
I/O cost: $N^C + (|C| \times N^P) = 300 + (20.000 \times 250) = 5.000.300$

The logical plan does not
tell us which one should
be outer and which should
be inner



**We should always have
the smaller table as the outer table!**
Usually in terms of number of tuples



Naïve Nested Loop

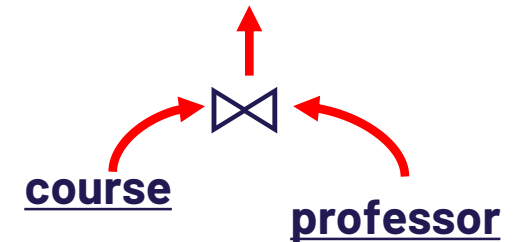
```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

```
foreach block  $B^P$  in P:  
  foreach tuple  $p$  in  $B^P$ :  
    foreach block  $B^C$  in C:  
      foreach tuple  $c$  in  $B^C$ :  
        if joinable(p,c):  
          emit (p,c)
```

Outer Table

Inner Table



The logical plan does not tell us which one should be outer and which should be inner

this will require $|P| \times |C|$ iterations.... but this is not the cost:

COST: For every tuple in P scan C
I/O cost: $N^P + (|P| \times N^C)$

This algorithm reads all the pages for every single tuple!



Block Nested Loop

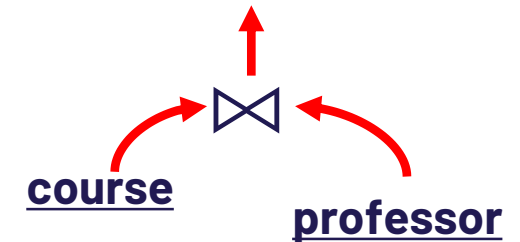
```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

*Remember:
block = page*

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

```
foreach block  $B^P$  in P:  
  foreach block  $B^C$  in C:  
    foreach tuple p in  $B^P$ :  
      foreach tuple c in  $B^C$ :  
        if joinable(p,c):  
          emit (p,c)
```

Now we take care of using all tuples in a block before throwing it away



Outer Table

Inner Table

COST: For every block in P scan every block in C
I/O cost: $N^P + (N^P \times N^C)$



Block Nested Loop (Cost)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

$|P| = 10.000, N^P = 150$
 $|C| = 500.000, N^C = 500$

COST: For every block in P scan blocks in C
I/O cost: $N^P + (N^P \times N^C) = 150 + (150 \times 500) = 75.150$

COST: For every block in C scan blocks in P
I/O cost: $N^C + (N^C \times N^P) = 500 + (500 \times 150) = 75.500$

The logical plan does not
tell us which one should
be outer and which should
be inner



**We should always have
the smaller table as the outer table!**
Based on the number of pages

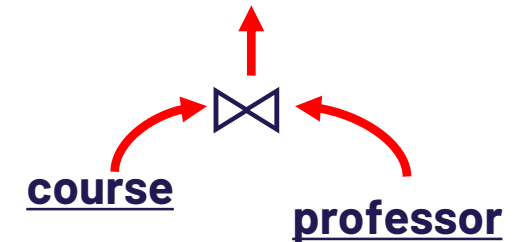
Outer Table

Inner Table

Block Nested Loop With M buffers

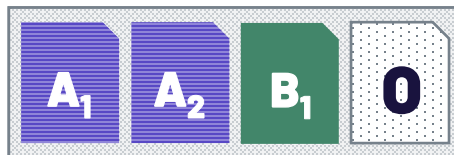
```
SELECT p.id, p.name, ....
FROM professor p
      JOIN course c ON p.id = c.by
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages
- **We have M buffers, $K = M - 2$** (1 for output, 1 for other table)



```
foreach blocks  $[B_1^P \dots B_K^P]$  in  $P$ :
    foreach block  $B^C$  in  $C$ :
        foreach tuple  $p$  in  $[B_1^P \dots B_K^P]$  :
            foreach tuple  $c$  in  $B^C$ :
                if joinable( $p, c$ ):
                    emit ( $p, c$ )
```

We keep a bigger
portion of the outer
table in memory



$M=4$
 $K=2$

If we have M buffers, we can use $K=M-2$ buffers to load in memory more pages of the outer table

COST: For every block in P scan every block in C
I/O cost: $N^P + ([N^P / K] \times N^C)$

Block Nested Loop With M buffers (Cost)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages
- **We have M buffers, $K = M - 2$**

$|P| = 10.000$, $N^P = 150$
 $|C| = 500.000$, $N^C = 500$
 $M=5$ $K=3$

COST: For every block in P scan blocks in C

I/O cost: $N^P + (N^P/3 \times N^C) = 150 + (150/3 \times 500) = 25.150$

COST: For every block in C scan blocks in P

I/O cost: $N^C + (N^C/3 \times N^P) = 500 + (500/3 \times 150) = 25.550$

The logical plan does not
tell us which one should
be outer and which should
be inner



**We should always have
the smaller table as the outer table!**
Based on the number of pages



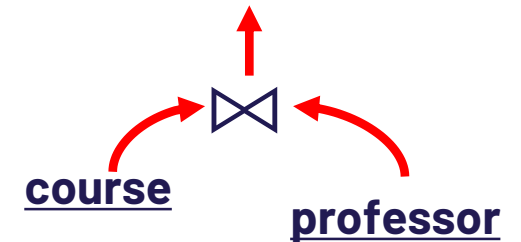
Index Nested Loop

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation as $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

```
foreach block  $B^P$  in P:  
    foreach tuple p in  $B^P$ :  
        foreach tuple c in IndexSearch(p,C):  
            if joinable(p,c):  
                emit (p,c)
```

index search can
return multiple tuples



If one relation has a compatible index, is convenient to use it in the inner loop in place of scanning the table

How much is the cost of IndexSearch? It depends! We assume some constant Q

COST: For every tuple in P search the index for C
I/O cost: $N^P + (|P| \times Q^C)$

We should always have the smaller table as the out table!
Based on the number of tuples!



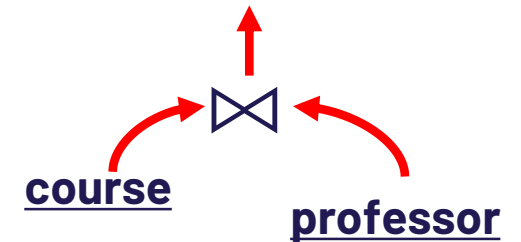
Sort-Merge Join

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

1. sort(P) & sort(C)
2. start reading first page B^P and B^C
3. get tuple p in B^P and c in B^C
4. check if joinable(p,c): emit (p,c)
5. advance cursors for B^P or B^C or both
6. repeat from 3.

These steps are missing the backtracking
for duplicate keys!



**We exploit the same properties of the
sort-merge algorithm**

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200



id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

p1.id	p2.id	p1.salary	p2.salary

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200



p1.id	p2.id	p1.salary	p2.salary

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200



p1.id	p2.id	p1.salary	p2.salary

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200



p1.id	p2.id	p1.salary	p2.salary

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```

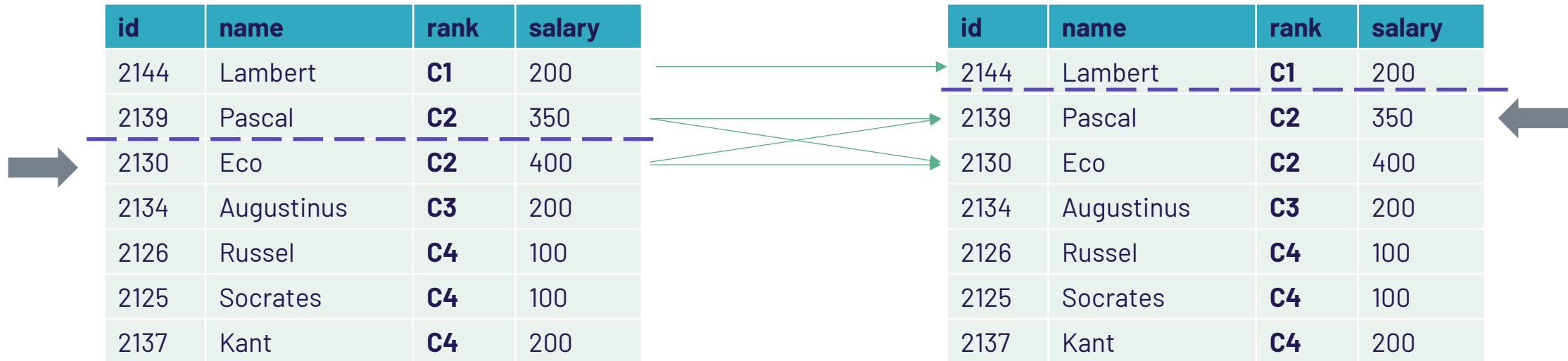
id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

id	name	rank	salary
2144	Lambert	C1	200
2139	Pascal	C2	350
2130	Eco	C2	400
2134	Augustinus	C3	200
2126	Russel	C4	100
2125	Socrates	C4	100
2137	Kant	C4	200

p1.id	p2.id	p1.salary	p2.salary

Example Run

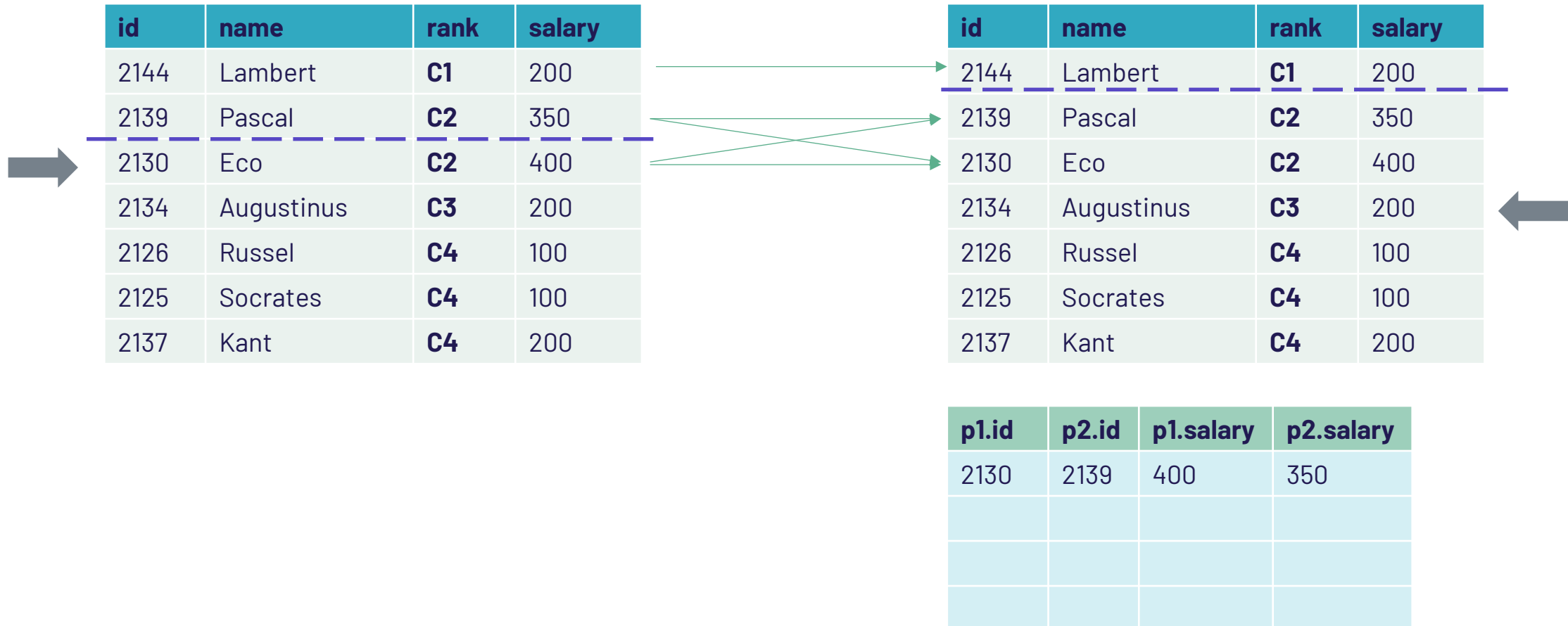
```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



p1.id	p2.id	p1.salary	p2.salary
2130	2139	400	350

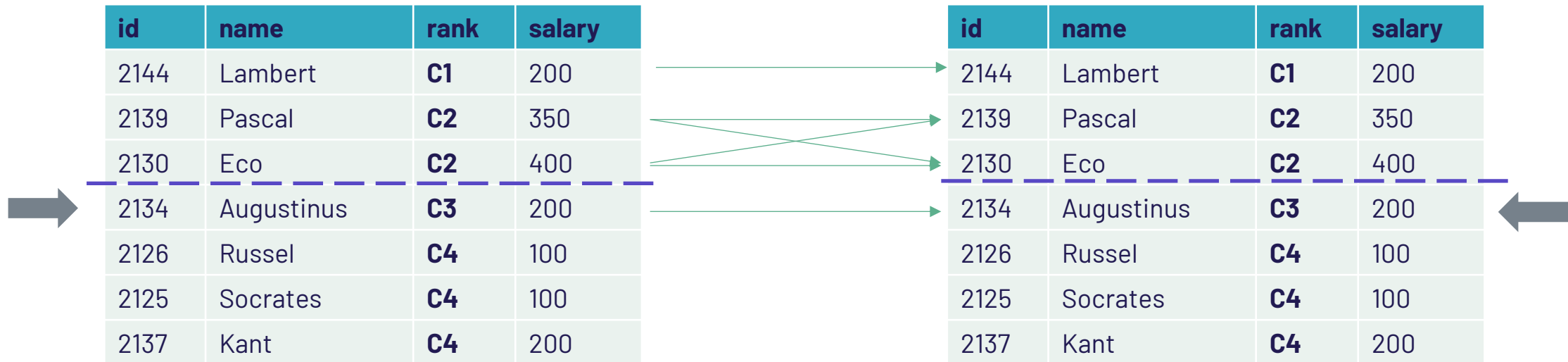
Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



ETC...

p1.id	p2.id	p1.salary	p2.salary
2130	2139	400	350

Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory ★

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1, 12.3, 12.6, 13.1–13.3, 14.1–14.3, 14.5

– from *Database System Concepts* 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

– <https://15445.courses.cs.cmu.edu/fall2022/>





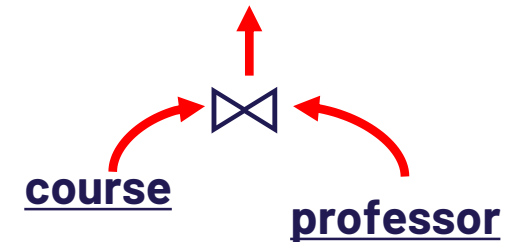
Sort-Merge Join

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

1. sort(P) & sort(C)
2. start reading first page B^P and B^C
3. get tuple p in B^P and c in B^C
4. check if joinable(p,c): emit (p,c)
5. advance cursors for B^P or B^C or both
6. repeat from 3.

These steps are missing the backtracking
for duplicate keys!

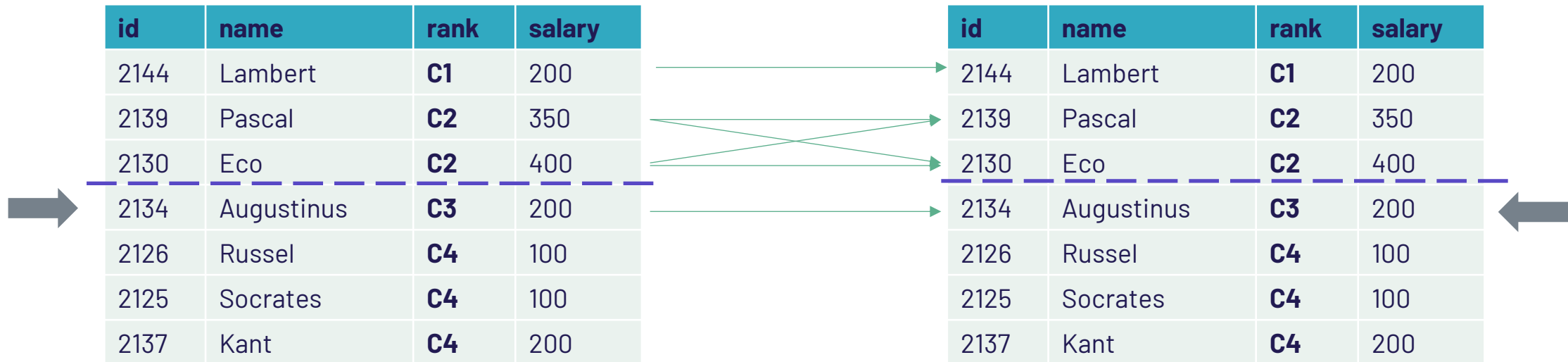


**We exploit the same properties of the
sort-merge algorithm**

Assume M buffers & External Merge Sort
COST: Sort P and C using $B=M-1$ buffers, scan P & C
I/O cost: $2*N^P \times (1+\log_B(N^P/B))$
 $+ 2*N^C \times (1+\log_B(N^C/B))$
 $+ N^P + N^C$

Example Run

```
SELECT p1.id, p2.id, p1.salary , p2.salary
FROM professor p1
      JOIN professor p2
      ON p1.rank = p2.rank
WHERE p1.salary > p2.salary
```



ETC...

p1.id	p2.id	p1.salary	p2.salary
2130	2139	400	350

Sort-Merge Joins With M buffers (Cost)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages
- **We have M buffers, $B = M - 1$**

Remember:

We also round by "CEILING" the fractions and logarithms!

Optimistic!

$|P| = 10.000$, $N^P = 150$
 $|C| = 500.000$, $N^C = 500$
 $M=5$ $B=4$

COST: Sort P and C using $B=M-1$ buffers, scan P & C

$2 * 150 \times (1 + \log_4(150/4)) + 2 * 500 \times (1 + \log_4(500/4)) + 150 + 500 = 300 \times 4$
 $+ 1000 \times 6 + 650 = 7850$

There is no concept of
outer table!

If the join attribute in one or both tables have a lot of duplicate keys, we pay a great cost for backtracking

If the join attribute has the same value in all tuples, the cost is Sort + $N^P \times N^C$



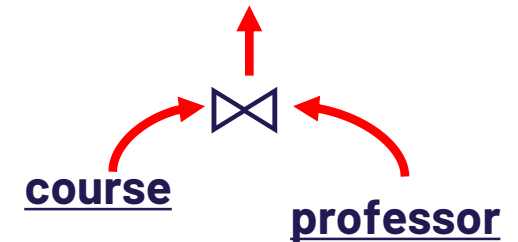


Sort-Merge Join

It's useful when:

- one or both tables are already sorted on the join attribute
- output must be sorted on the join attribute
- the join attribute contains a small number of duplicate values in one of the two relations
- both tables are very large
- cost of sorting is reduced (e.g., scanning an index)

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```



We exploit the same properties of the sort-merge algorithm

Assume M buffers & External Merge Sort
COST: Sort P and C using $B=M-1$ buffers, scan P & C
I/O cost: $2 * N^P \times (1 + \log_B(N^P/B))$
 $+ 2 * N^C \times (1 + \log_B(N^C/B))$
 $+ N^P + N^C$

Index Nested Loop

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

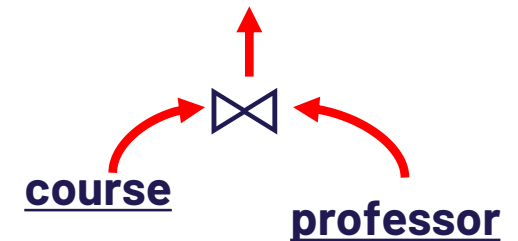
- We have two relations P and C
- each relation has $|P|$ and $|C|$ tuples
- each relation is stored in N^P and N^C pages

```
foreach block  $B^P$  in P:  
    foreach tuple p in  $B^P$ :  
        foreach tuple c in IndexSearch(p,C):  
            if joinable(p,c):  
                emit (p,c)
```

index search can
return multiple tuples

we want an index
with a small
cost Q

How much is the cost of IndexSearch? It
depends! We assume some constant Q



If one relation has a compatible index, is convenient to use it in the inner loop in place of scanning the table

What if we don't have an index?
We build it!





Hash Join: build a temporary hash index

When joining on attributes like **Foreign Keys** then tuples p and c have **the same value for the join attribute**

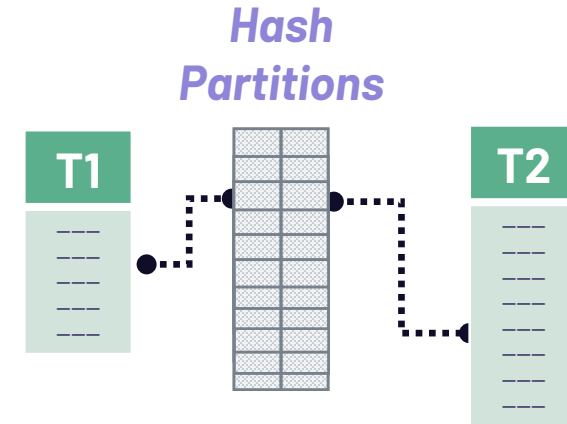
Then we can hash that value!

Tuples are stored in “partitions” based on the hash of their value.

Only tuples in the same partition need to be compared

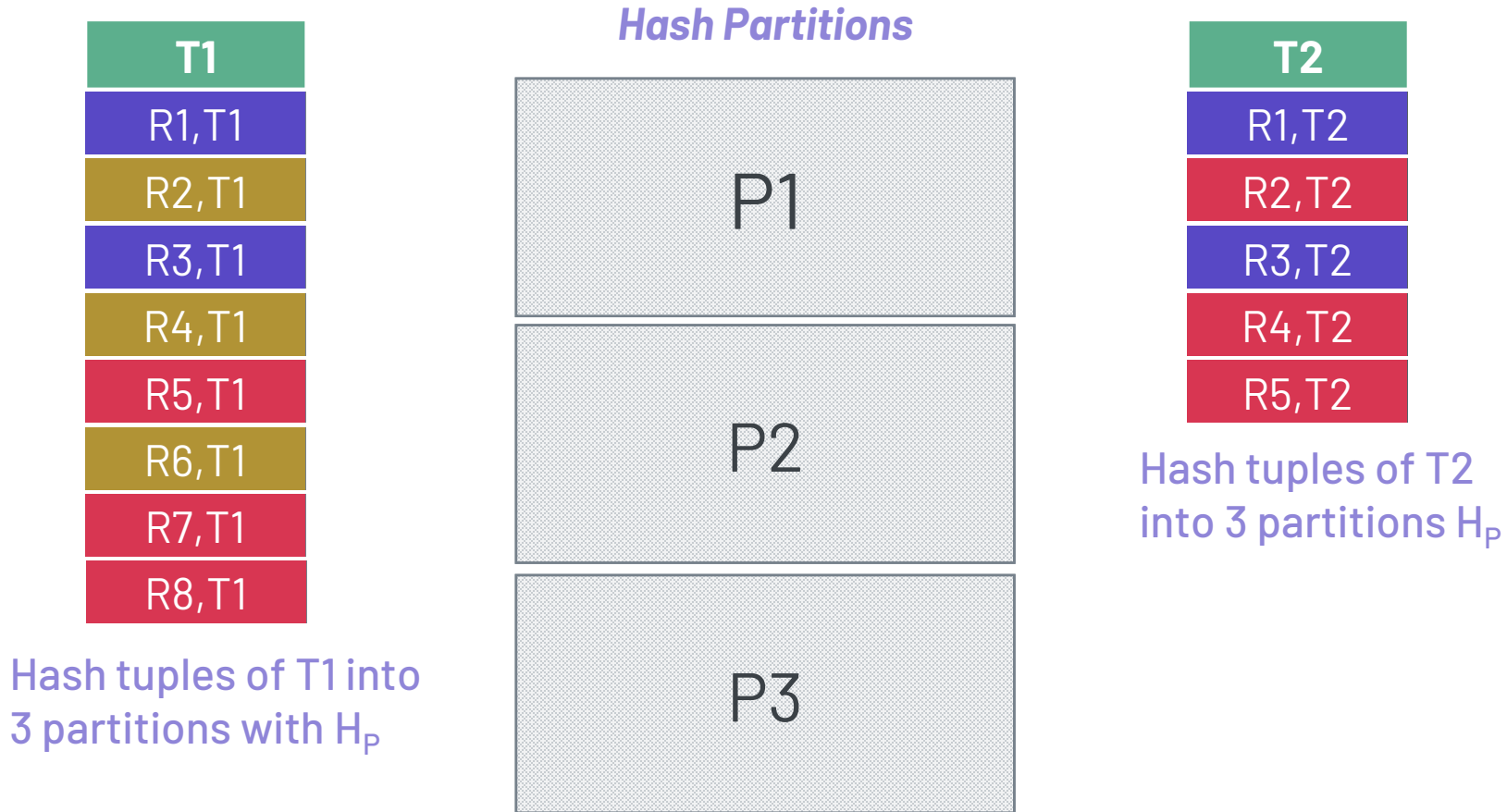
Overview:

1. Split T1 in M partitions, split T2 in M partitions
2. Load one partition P of T1
3. Build Hash Index on P
4. Iterate over all pages for same partition of T2
5. Hash key of tuple from partitions of T2
6. Test join condition with tuples stored in the corresponding hash index slot
7. When done, move to next partition



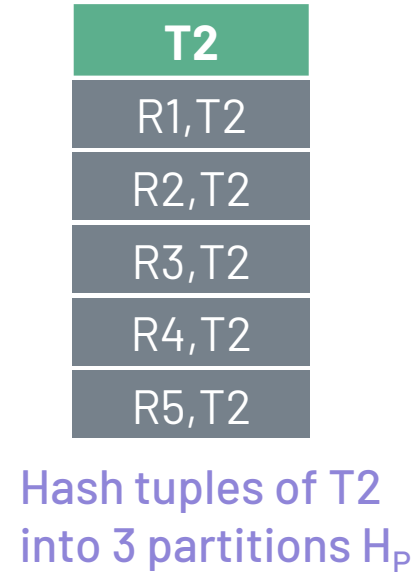
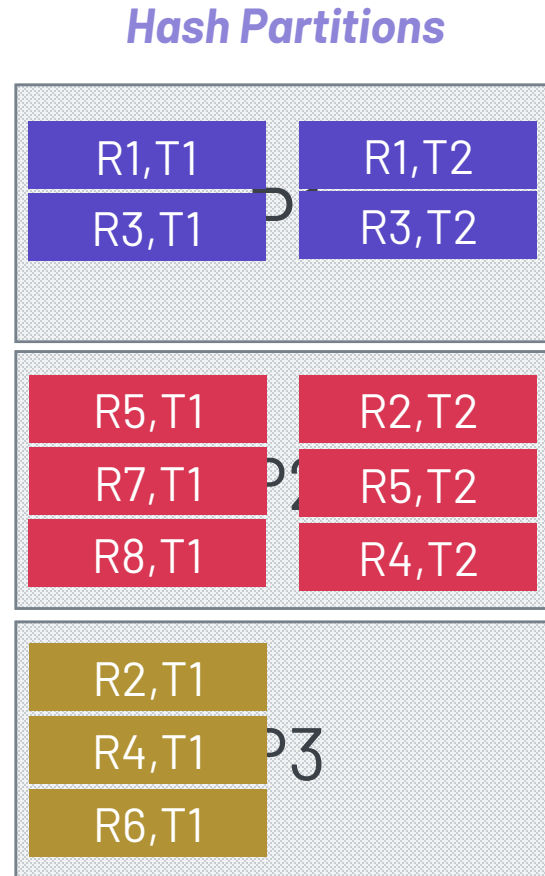
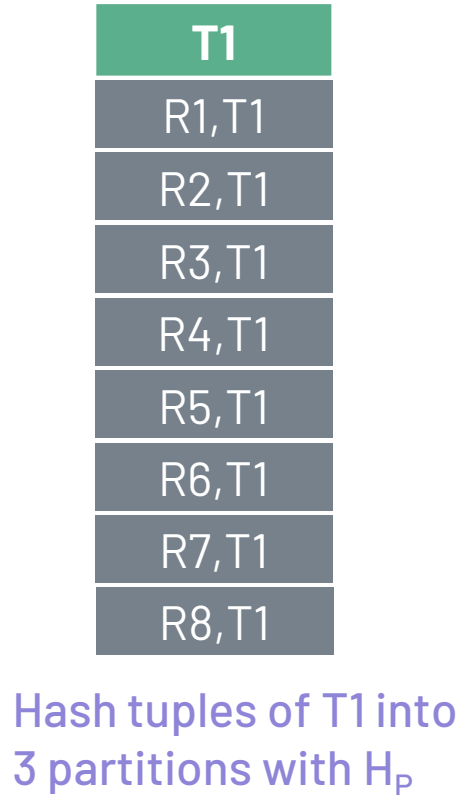


Hash Join: example





Hash Join: example



Hash Index for P1 of T2



Needs to use a different function than H_p !!!

With M buffers, each partition+index needs to fit into $B=M-2$ buffers



COST: depends on many factors.

$2(N^{T1} + N^{T2})$ to build partitions. Then each partition is read only once so is and additional $(N^{T1} + N^{T2})$. But this is for perfect hashing...

If there is skew, we will do recursive hash partitioning...



Query Plan & Query Execution

SQL queries are declarative

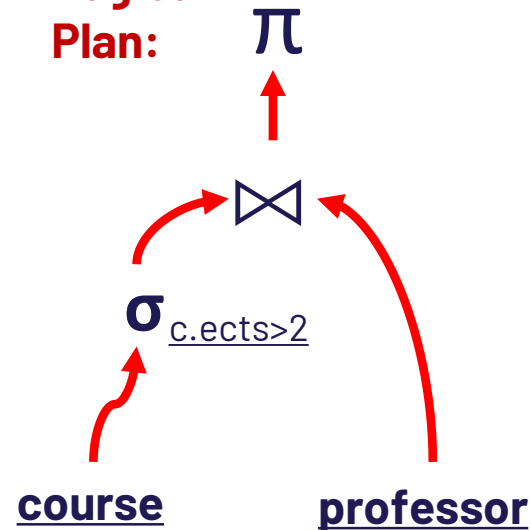
The DBMS needs to translate them into a series of operations to compute the correct result

Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Declarative

Logical Plan:



Physical Plan:

Hash Join

Hash Cond: (c.by = p.id)

-> Seq Scan on course c
Filter: (ects > 2)

-> Hash
-> Seq Scan on professor p

Procedural

Data flows from the bottom (leaves of the plan tree) to the top (root)

Outline

1. Query Plan & Execution

- ▶ Relational Algebra Equivalent ★
- ▶ Dataflow
- ▶ Logical plan – physical plan ★
- ▶ Intermediate Result do not fit in memory

2. Sorting Algorithms

- ▶ When sorting is needed ★
- ▶ External Memory sorting ★
- ▶ External merge sort ★

3. Grouping Aggregation Algorithms

- ▶ Sorting
- ▶ Hashing ★
- ▶ Computing Distinct & Aggregations

4. Join Algorithms

- ▶ Nested Loop Join ★
- ▶ Sort-Merge Join ★
- ▶ Hash Join ★
- ▶ Index Nested Loop Join
- ▶ Early/Late materialization
- ▶ Cost of Join ★
- ▶ Join vs. Cross product

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

12.1-12.4, 13.2-13.4, 14.1-14.6, 15.4-15.6

- from Database System Concepts 7th edition ; Silberschatz et al.

Intro to Database Systems (15-445/645), Andy Pavlo CMU

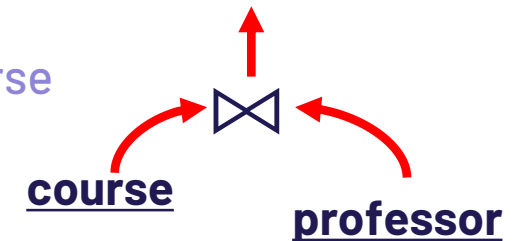
- <https://15445.courses.cs.cmu.edu/fall2022/>

Hash Join Optimization

```
SELECT p.id, p.name, ....  
FROM professor p  
      JOIN course c ON p.id = c.by  
WHERE ...
```

- For every tuple in the outer table, we need to search the hash index
- This is fast, but not free if we have many redundant keys or many keys that do not join
- **Idea:** use something **faster than a hash index search** to know when we can skip the search

Assume we build hash index
on course but only $\frac{1}{4}$ of
professors teach a course



```
foreach block BP in P:    if this returns False  
                           we don't search the index  
    foreach tuple p in BP:  
        if IndexContains(p,C):  
            foreach tuple c in IndexSearch(p,C):  
                if joinable(p,c):  
                    emit (p,c)
```

IndexContains must be
faster than IndexSearch !!

Since we are building the
Hash Index, we can also build
some auxiliary data structure
to speed up the search



JOIN COST RECAP

Algorithm	I/O Cost	Example
Simple Nested Loop Join	$N^0 + (O \cdot N^i)$	1.3 hours
Block Nested Loop Join	$N^0 + (N^0 \cdot N^i)$	50 seconds
Index Nested Loop Join	$N^0 + (O \cdot Q)$	Variable
Sort-Merge Join	$N^0 + N^i + (\text{sort cost})$	0.75 seconds
Hash Join	$3 \cdot (N^0 + N^i)$ (*only in ideal conditions)	0.45 seconds

Pages Outer Table N^0
Tuples Outer Table $|O|$
Pages Inner Table N^i
Index Access Cost Q



- For each query multiple logical plans and multiple physical plans can exist
- Each operator needs to process a large amount of data and this can be more than what fits in main memory
- Sorting can be useful in many situations, not just when we use ORDER BY
- Joining is the core operation in the relational model
- We need to focus on minimizing I/O, especially the amount of times we read from disk
- We can scan pages and optimize # block read, but better if we can be smarter, e.g., sort tuples and use merge sort, exploit an existing index, or even build one on the fly
- The job of the DBMS is to “choose wisely”



- For each query multiple logical plans and multiple physical plans can exist
- Each operator needs to process a large amount of data and this can be more than what fits in main memory
- Sorting can be done in memory or on disk
 - use ORDER BY
- Joining is the most expensive operation
 - We need to find the best join order
 - of times we need to join
- We can scan a table or use an index
 - if we can be sure that the index is the best
 - exploit an existing index, or even build one on the fly
- The job of the DBMS is to “choose wisely”



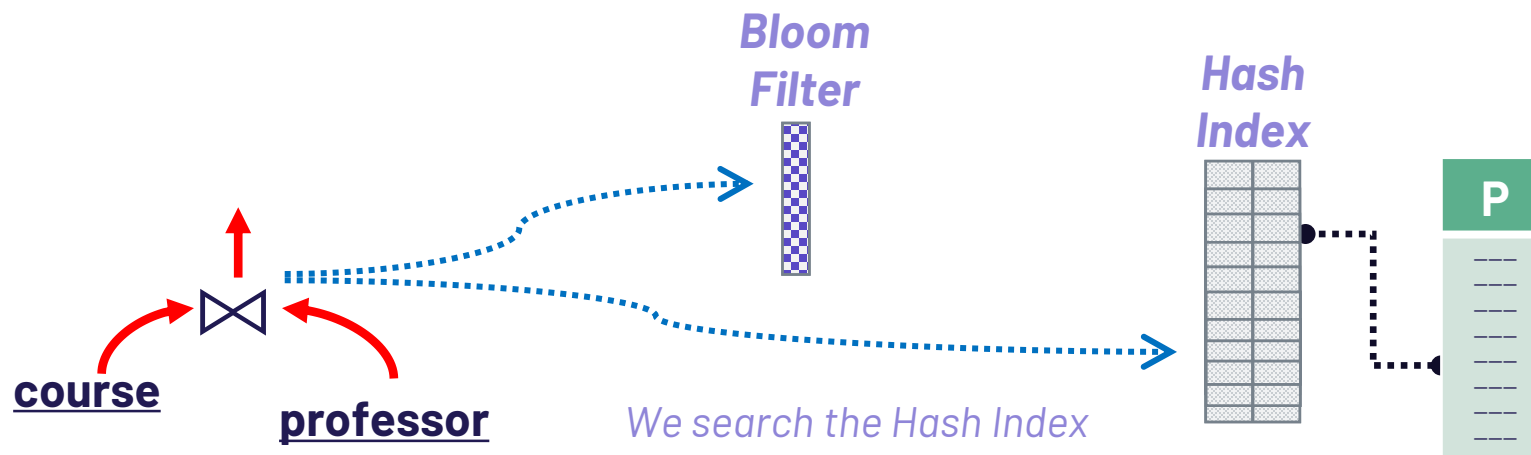
Bloom Filter: Probabilistic Membership

Bloom Filter is a probabilistic data structure for “membership check”

Given a set of elements P , **BloomFilter(P)** is a datastructure with 2 methods: PUT & CONTAINS

Given an element p_1 in P then **BloomFilter(P).contains(p_1) will return True**

Given an element p_2 NOT in P then **BloomFilter(P).contains(p_2) will return False with high probability, but sometimes will return True (so True is *actually a “Maybe”!*)**



The only thing faster than a hash index, is a simpler hash index!

Bloom Filter: Example

Bloom Filter is a probabilistic data structure for “membership check”

Prepare a **binary fingerprint F** of size N set to all 0 (**zeroes**)

Use $K \ll N$ hash functions.

Insert: compute K hash functions, each one correspond **to K bits to set to 1 in F**

Contains: compute K hash functions, return **Maybe if all K are set to 1** otherwise return False

3 Hash Functions: H_1, H_2, H_3

Insert T123: $H_1(T123)=1$; $H_2(T123)=3$; $H_3(T123)=4$

Bloom Filter

0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0

Bloom Filter: Example

Bloom Filter is a probabilistic data structure for “membership check”

Prepare a **binary fingerprint F** of size N set to all 0 (**zeroes**)

Use $K \ll N$ hash functions.

Insert: compute K hash functions, each one correspond **to K bits to set to 1 in F**

Contains: compute K hash functions, return **Maybe if all K are set to 1** otherwise return False

3 Hash Functions: H_1, H_2, H_3

Insert T123: $H_1(T123)=1$; $H_2(T123)=3$; $H_3(T123)=4$

Insert T235: $H_1(T235)=1$; $H_2(T235)=0$; $H_3(T235)=5$

Bloom Filter

0	1	2	3	4	5	6	7
0	<u>1</u>	0	<u>1</u>	<u>1</u>	0	0	0

Bloom Filter: Example

Bloom Filter is a probabilistic data structure for “membership check”

Prepare a **binary fingerprint F** of size N set to all 0 (**zeroes**)

Use $K \ll N$ hash functions.

Insert: compute K hash functions, each one correspond **to K bits to set to 1 in F**

Contains: compute K hash functions, return **Maybe if all K are set to 1** otherwise return False

To choose parameters to reduce the probability of false positives: <https://hur.st/bloomfilter/>

3 Hash Functions: H_1, H_2, H_3

Insert T123: $H_1(T123)=1$; $H_2(T123)=3$; $H_3(T123)=4$

Insert T235: $H_1(T235)=1$; $H_2(T235)=0$; $H_3(T235)=5$

Contains T111: $H_1(T111)=1$; $H_2(T111)=2$; $H_3(T111)=5$: **False**

Contains T123: $H_1(T123)=1$; $H_2(T123)=3$; $H_3(T123)=4$: **Maybe**

Contains T222: $H_1(T222)=5$; $H_2(T222)=1$; $H_3(T222)=4$: **Maybe!**

Bloom Filter

0	1	2	3	4	5	6	7
<u>1</u>	<u>1</u>	0	1	1	<u>1</u>	0	0

X

A Bloom filter avoids searching for something that is not there!



DBMS Internals

- ▶ We know how to use a DBMS.
- ▶ We know how to store and access data efficiently in a DBMS.
- ▶ We know **how to execute queries efficiently**.
- ▶ How do we **make sure we run the query** in the **most efficient way** among the alternatives available?

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager



Query Plan & Query Execution

SQL queries are declarative

The DBMS needs to translate them into a series of operations to compute the correct result

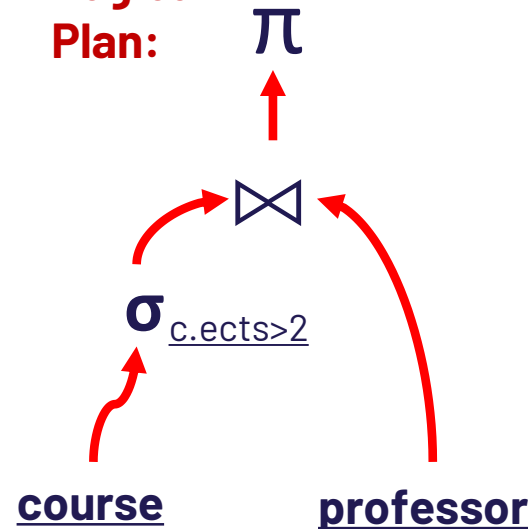
Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Declarative

How do we translate this to actual machine code?

Logical Plan:



Physical Plan:

Hash Join

Hash Cond: (c.by = p.id)

-> Seq Scan on course c

Filter: (ects > 2)

-> Hash

-> Seq Scan on professor p

We need to “**connect**” the output of **each operator** to the input of the next and **decide how information flows in practice!**



Query Processing/Execution Models

How do we actually execute a physical plan?

This will have a real effect on I/O, especially if it causes to materialize large intermediate results

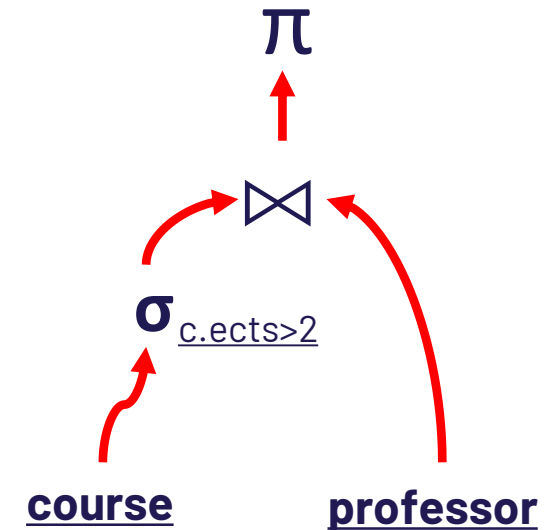
Alternatives:

1. Iterator / Volcano / Pipeline Model

2. Materialization Model

3. Vectorized / Batch Model

Logical Plan $\Rightarrow$ Physical Plan



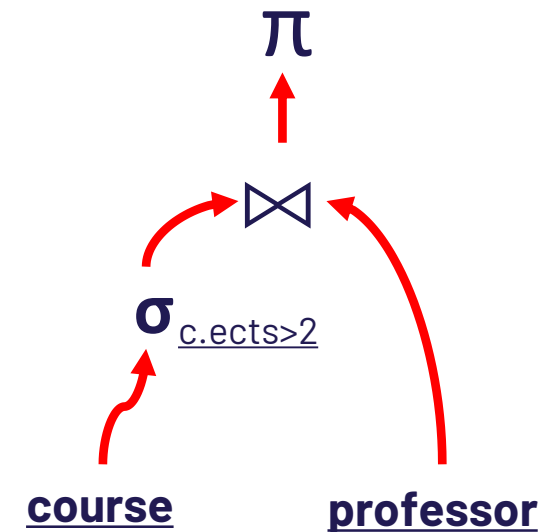


1. Iterator / Volcano / Pipeline Model

Each query operator is seen as an “iterator” with a `next()` method

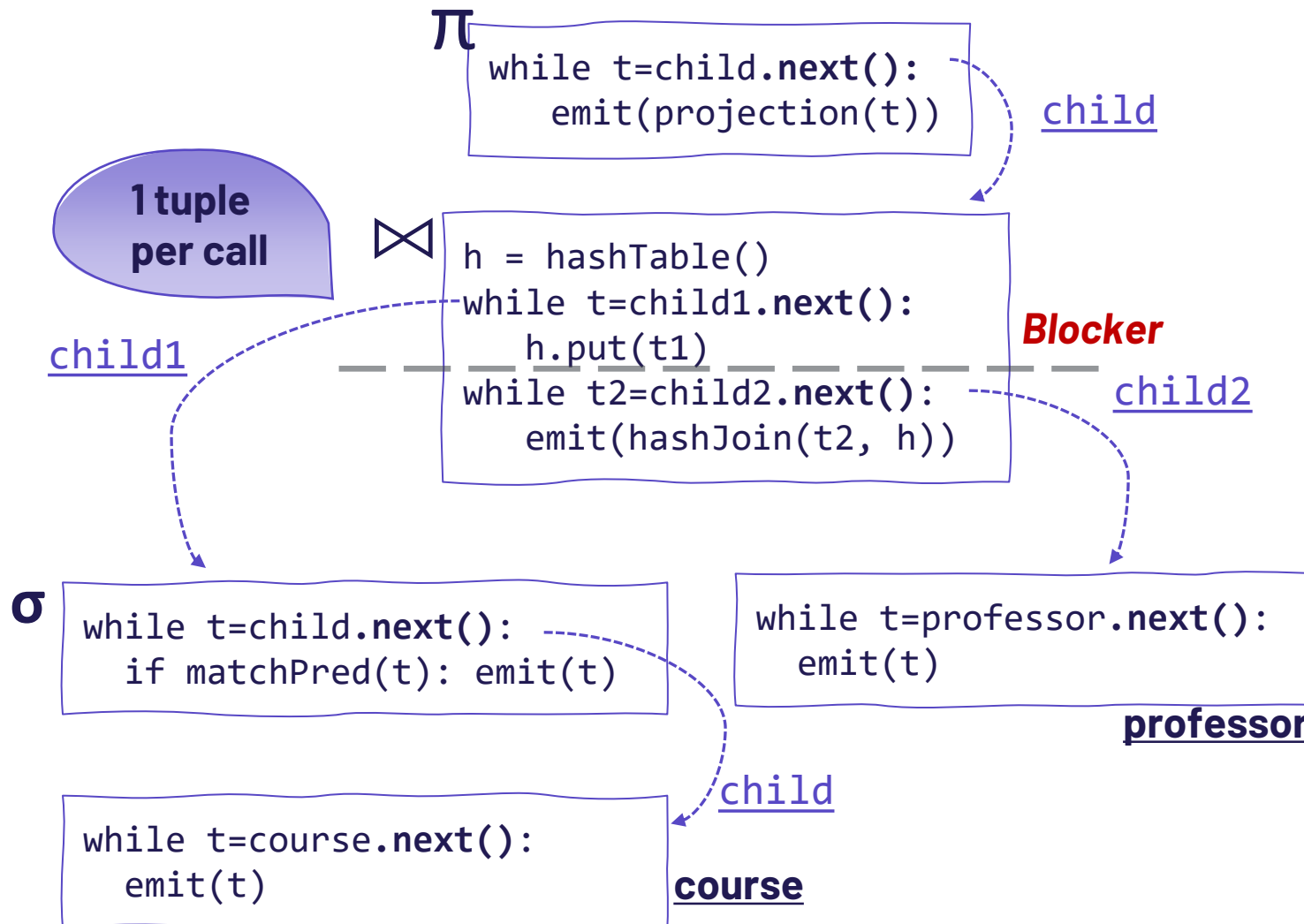
- The method returns either a single tuple or “null” if there are no more tuples
- Each operator has to call `next()` on its children to process their output
- This is a top-down execution approach
- The parent does not know how long it takes to produce the “next tuple”

Logical Plan $\Rightarrow$ Physical Plan





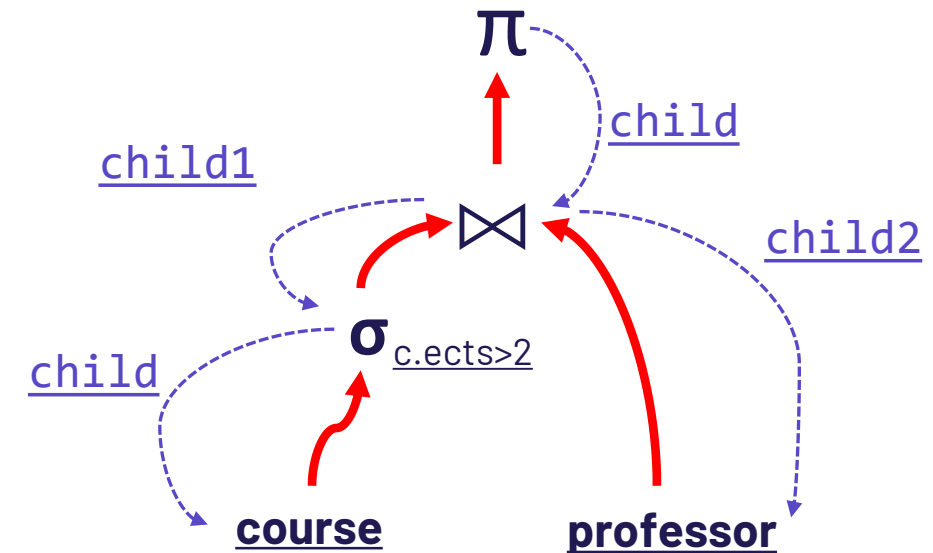
1. Iterator Model Example



Query:

```
SELECT p.name, c.title, c.ects  
FROM course c JOIN professor p  
      ON p.id = c.taughtby  
WHERE c.ects > 2;
```

Logical Plan:





1. Iterator / Volcano / Pipeline Model

Each query operator is seen as an “iterator” with a `next()` method

- The method returns either a single tuple or “null” if there are no more tuples
- Each operator has to call `next()` on its children to process their output
- This is a top-down execution approach
- The parent does not know how long it takes to produce the “next tuple”

**This is used in almost every DBMS. Allows for tuple pipelining
→ reduce materialization of intermediate results.**

Some operators must block until their children emit all their tuples!

e.g., Joins, Subqueries, Order By



2. Materialization Model

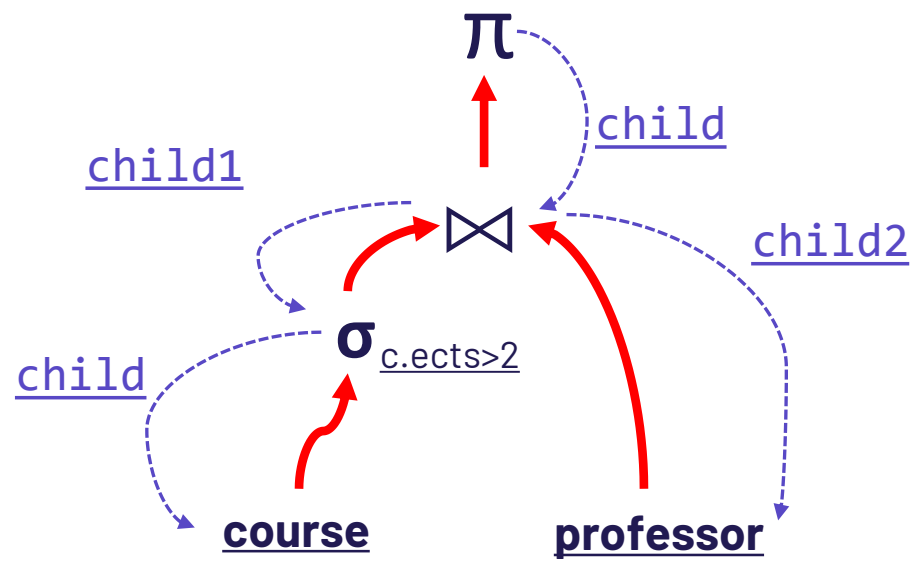
Each query operator processes all its input and produced all its output in one go

- **The operator “materializes” its output as a single result**
- **The DBMS should “push down” any possible optimization when possible (e.g., LIMIT, FILTER)**
- The output can be a subset of columns

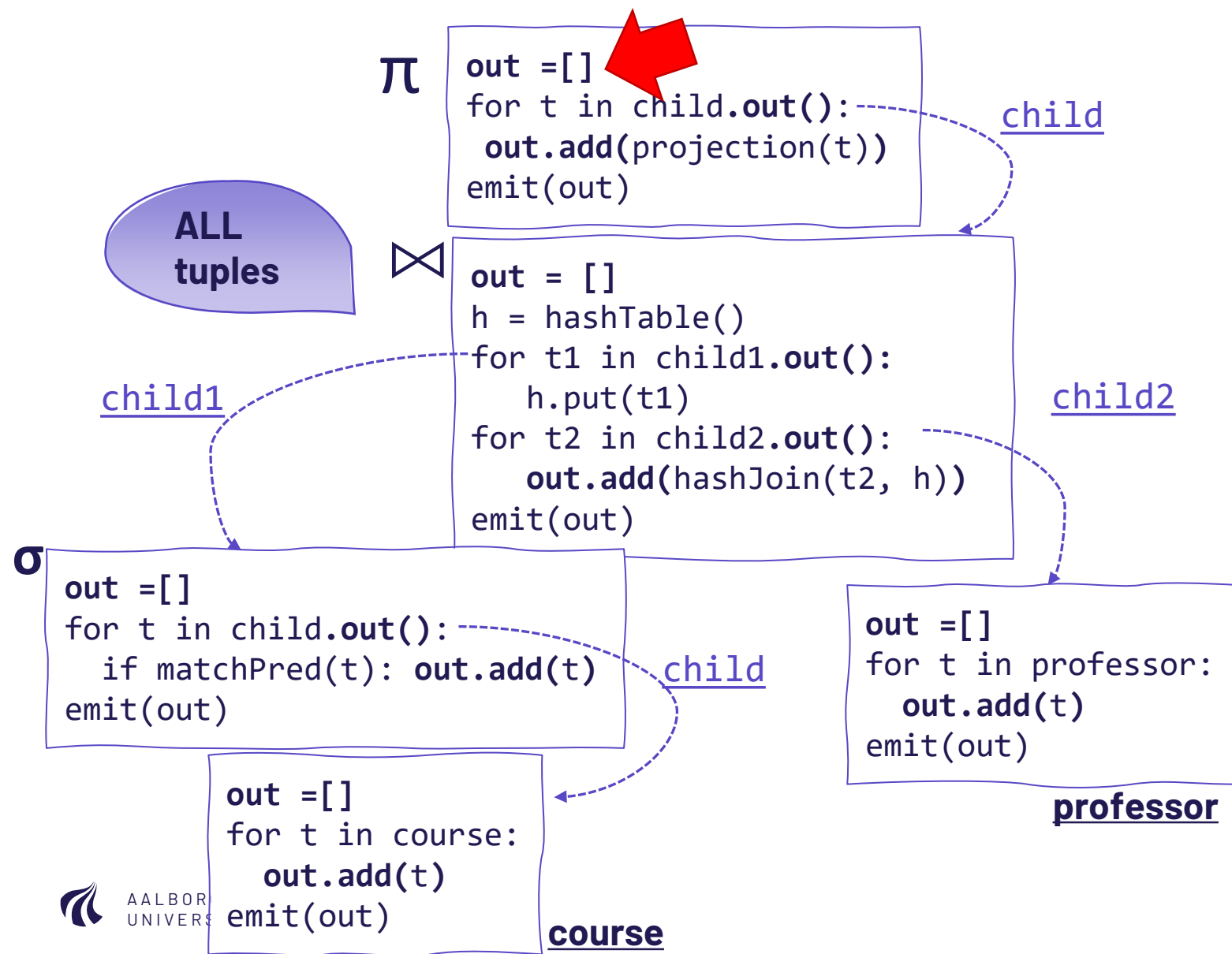
Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Logical Plan:



2. Materialization Model



2. Materialization Model

Each query operator processes all its input and produced all its output in one go

- The operator “materializes” its output as a single result
- The DBMS should “push down” any possible optimization when possible (e.g., LIMIT, FILTER)
- The output can be a subset of columns

This works best when we support queries with very small resultsets (OLTP)

- Lower execution/coordination overhead
- Fewer function calls

For columnstores this can be convenient,
but we need to ensure to minimize writing to disk



VOLTDB

RAVENDB

monetdb

+ CrateDB



3. Vectorization/Batch Model

Similar to the Iterator: **Each query operator is seen as an “iterator” with a `next()` method**

BUT each call emits a “batch” of tuples instead of a single tuple

- **The operator has an internal loop that “materializes” a batch of tuples at each iteration**
- **The size of the batch is a parameter optimized for hardware or for the specific query**

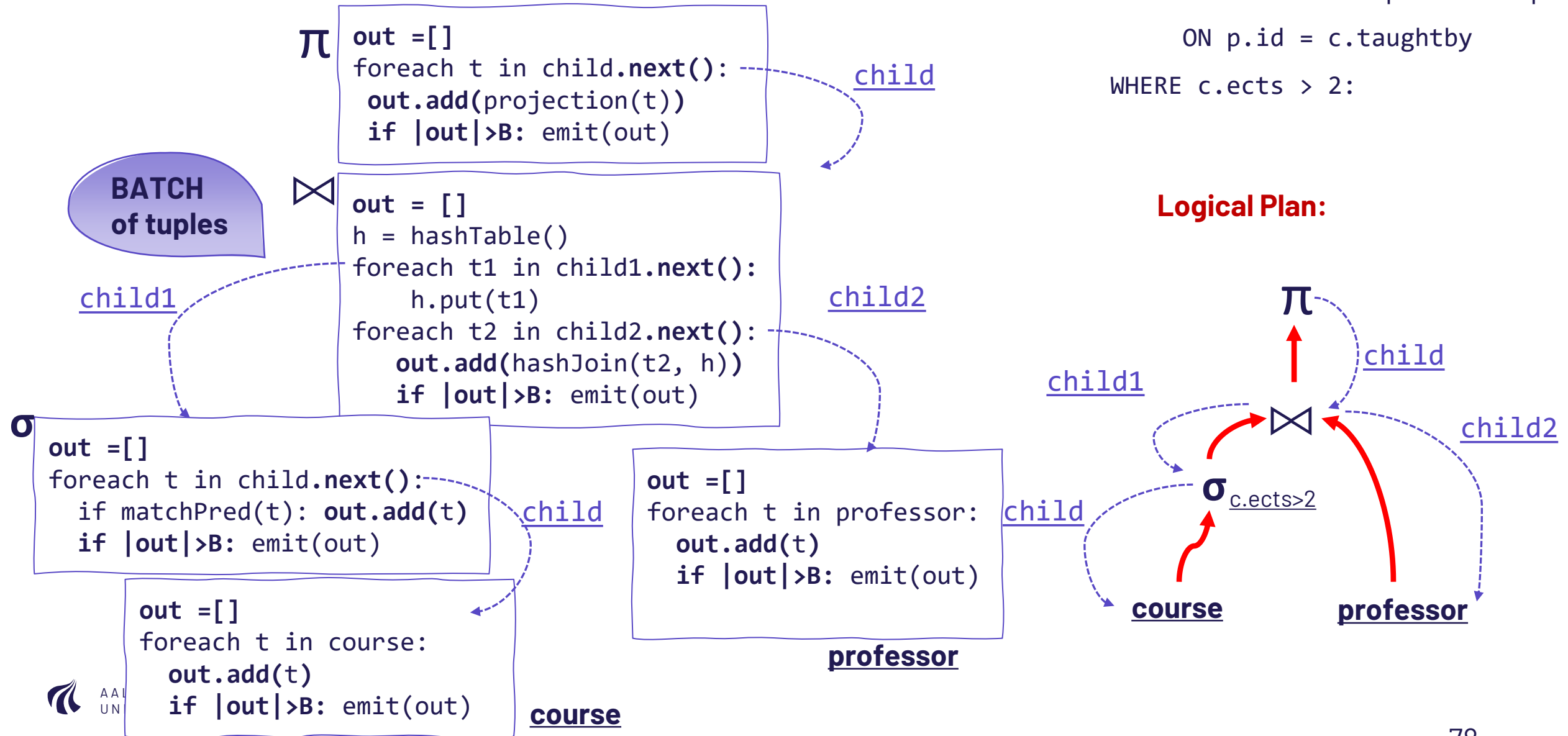


3. Batch Model

Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Logical Plan:





3. Vectorization/Batch Model

Similar to the Iterator: **Each query operator is seen as an “iterator” with a `next()` method**

BUT each call emits a “batch” of tuples instead of a single tuple

- **The operator has an internal loop that “materializes” a batch of tuples at each iteration**
- **The size of the batch is a parameter optimized for hardware or for the specific query**

This works best when we support analytical queries (OLAP)

- advantages of the iterator model: pipelining + low materialization cost
- Fewer function calls
- Allows to use advanced CPU instructions: SIMD to execute one operation in parallel on the batch





Access Method

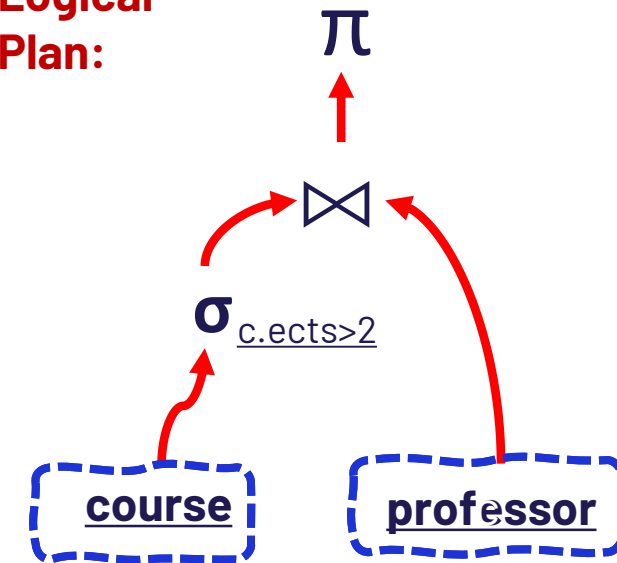
How do we get the initial tuples to “feed” the plan

The DBMS needs to access data stored in the table... or not...

Three basic approaches to reach tuple values:

- **Sequential Scan**
- **Index Scan** (depending on which values are stored)
- **Multi-Index Scan**

Logical Plan:



Physical Plan:

Hash Join

~~Hash Cond: (c.by = p.id)~~

-> Seq Scan on course c

~~Filter: (ects > 2)~~

-> Hash

-> Seq Scan on professor p

Index Scan: which index?

Suppose that we have a single table with 1000 tuples and two indexes:

- Index #1: age
- Index #2: dept

Query:

```
SELECT * FROM student  
WHERE age > 30  
AND dept= "CS"  
AND country="US"
```

Scenario #1

There are 990 people over the age of 30 but only 2 people in the CS department.

Scenario #2

There are 990 people in the CS department but only 2 people over the age of 30.

Pick the index with the **lowest selectivity**: the one that contains less tuples matching the condition

Multi-Index Scan: which index? both!

If there are multiple indexes that the DBMS can use:

1. Compute sets of Record IDs using each matching index;
2. Combine these sets based on the query's predicates (union vs. intersect);
3. Retrieve the records and apply any remaining predicates.

Query:

```
SELECT * FROM student  
WHERE age > 30  
AND dept= "CS"  
AND country = "US"
```

Suppose that we have a single table with 1000 tuples and two indexes:

→ Index #1: **age**

→ Index #2: **dept**

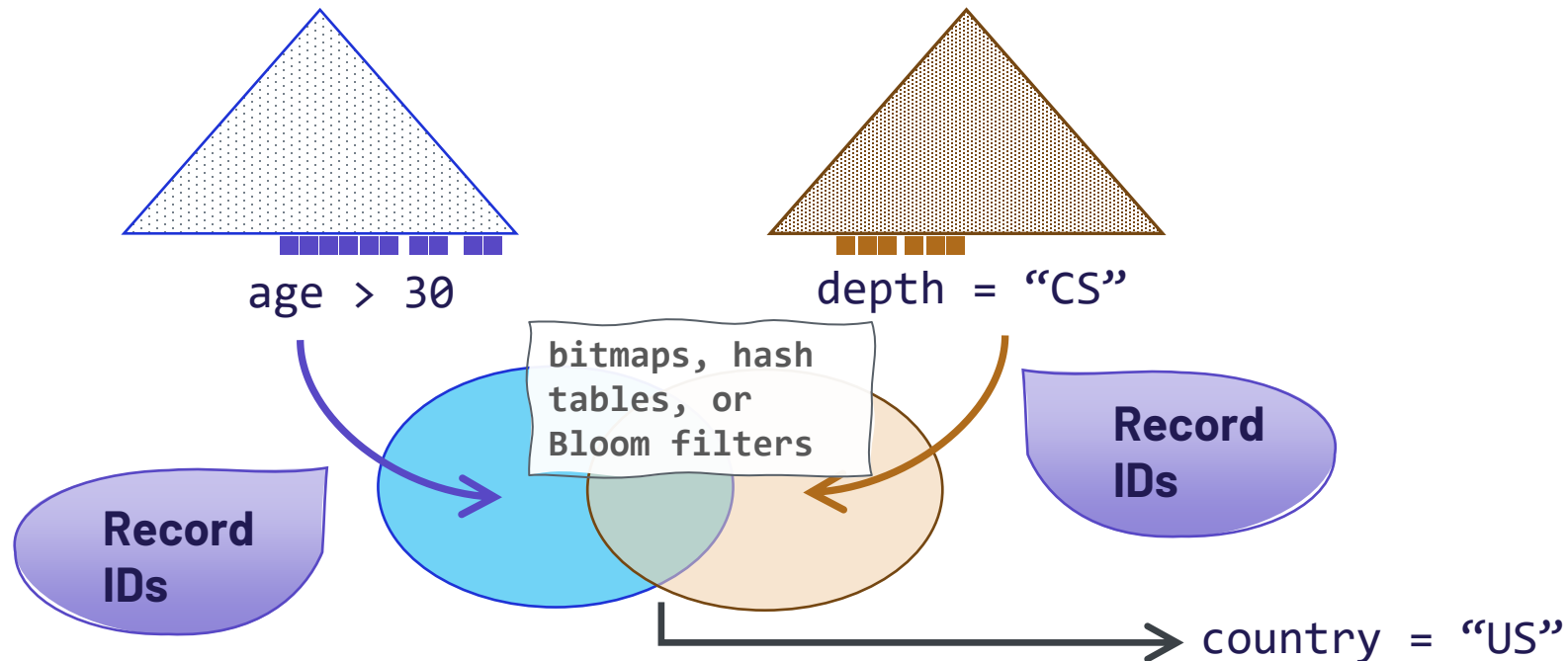
Scenario #3

There are 500 people in the CS department and 600 people over the age of 30.

Multi-Index Scan: set-based results

If there are multiple indexes that the DBMS can use:

1. Compute sets of Record IDs using each matching index;
2. Combine these sets based on the query's predicates (union for OR vs. intersect for AND) using the record id as items;
3. Retrieve the records and apply any remaining predicates.



Query:

```
SELECT * FROM student  
WHERE age > 30  
AND dept= "CS"  
AND country = "US"
```

Scenario #3

There are 500 people in the CS department and 600 people over the age of 30.

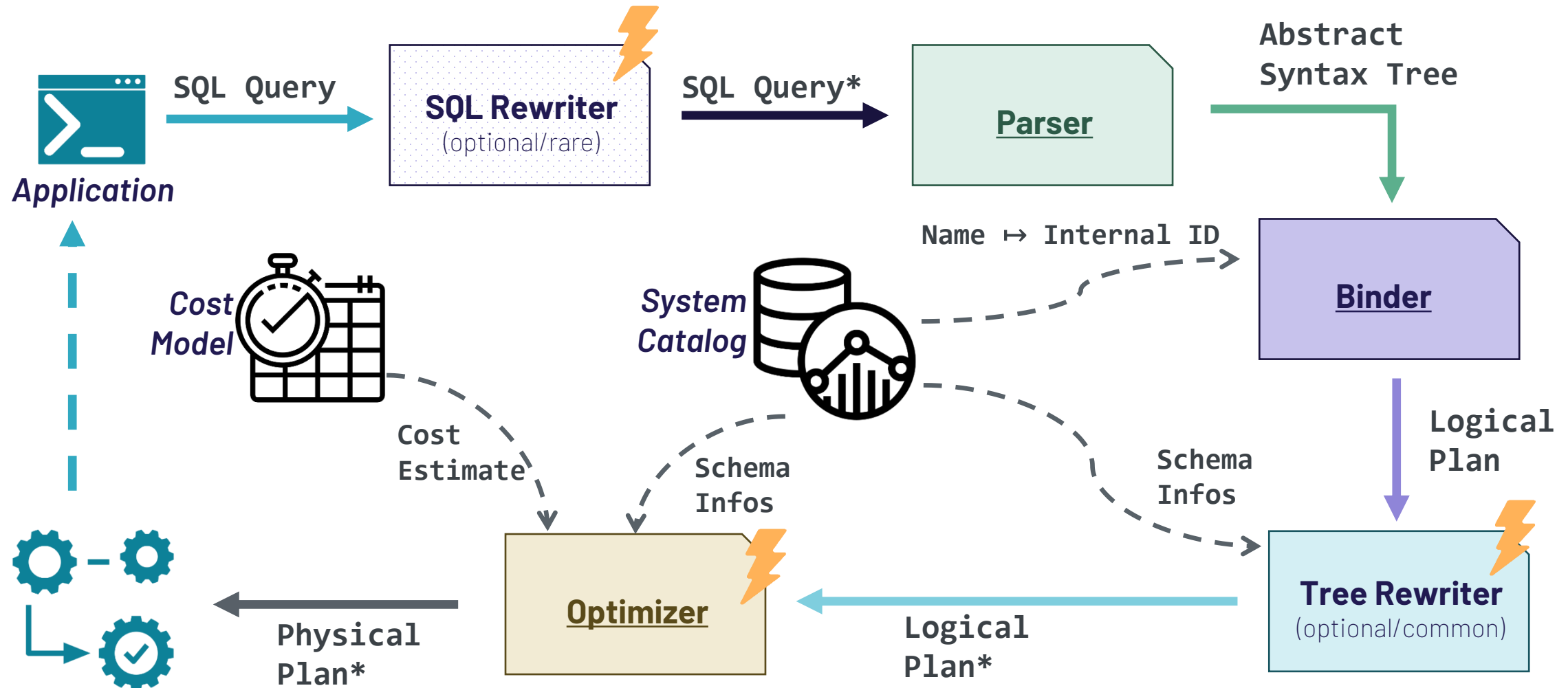
Examples:

- DB2 Multi-Index Scan
- PostgreSQL Bitmap Scan
- MySQL Index Merge



Query Execution Overview

From the SQL text to the code that computes the actual results





Query Optimization & Query Compilation

For a given query find a correct execution plan with the lowest cost

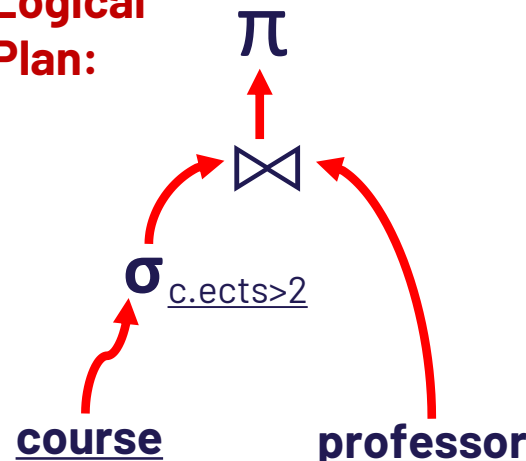
1. From the SQL query find an optimized logical plan
 - Optimize order of operators
2. From a given optimized logical plan find the most efficient Physical Plan
 - Select the best implementation for each operator

How do we translate a SQL query to the final actual machine code?

Query:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Logical Plan:



Physical Plan:

Hash Join

Hash Cond: (c.by = p.id)

-> Seq Scan on course c

Filter: (ects > 2)

-> Hash

-> Seq Scan on professor p

This problem is intractable (NP-Complete)

No optimizer produces always the "optimal" plan... often they guess



HINT:

When you ask yourself:

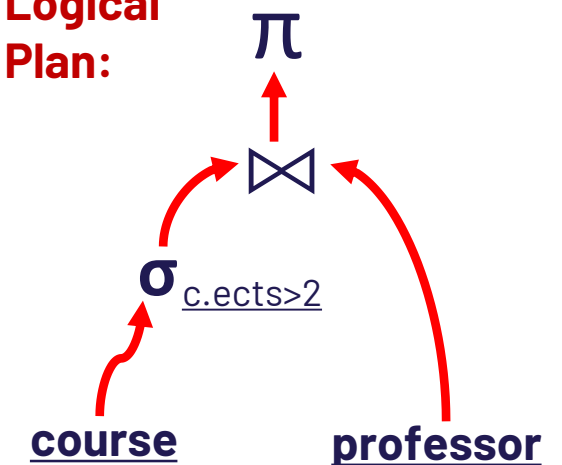
"How should I write the SQL query to?"

Try to build the Relational Algebra first, and then translate to SQL.

Query:

```
SELECT p.name, c.title, c.ects  
FROM course c JOIN professor p  
      ON p.id = c.taughtby  
WHERE c.ects > 2;
```

Logical Plan:





Query Optimization Methods

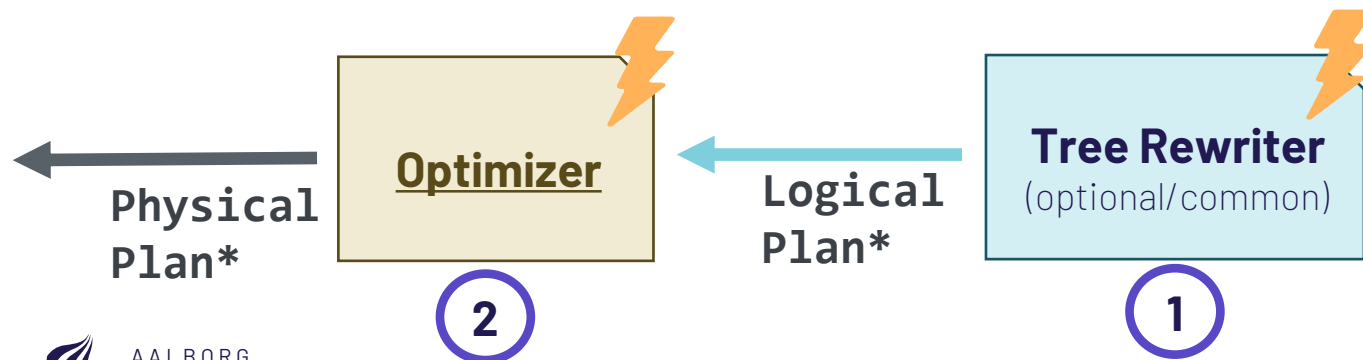
For a given query find a correct execution plan resulting in the fastest query

1. **Heuristics / Rules:** rewrite the query to remove or rewrite parts that are **obviously inefficient**

These techniques **may examine the catalog** (the schema) but not the data itself

2. **Cost-Based Search:** lowest “execution” cost → lowest running time

- Use a “model” to “estimate” the cost of executing a logical or physical plan (based on data)
- Enumerate **multiple equivalent** plans for the same query and pick the one with lowest cost



Most of the times
we cannot enumerate all the plans!

Yet, we guarantee to always return the correct result.



Cost of a Query Plan



- The final goal is to get to the answer as fast as possible: running time
- The big bottleneck (impacting running time) is disk I/O
- Large intermediate results can cause spill to disk!

Basic Principle:

The cost of the query plan is correlated with the amount of data processed and the resulting I/O to disk

- identify the execution plan that will require the least amount of I/O
- reduce the amount of data that is read, produced, and processed

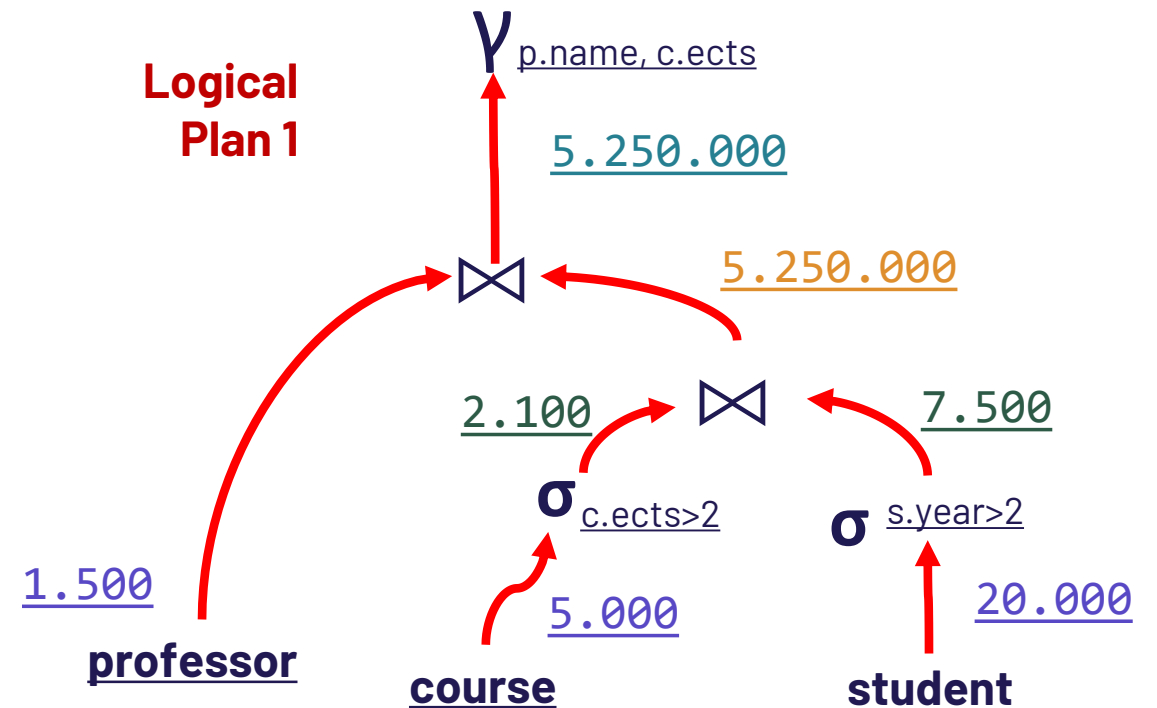
The DBMS uses a “cost model” to predict how “computationally expensive” a query plan will be: it’s an internal cost



Intermediate Results

The cost of the plan is correlated with the amount of data to process = # of tuples

```
SELECT p.name, c.ects, COUNT(*)
FROM course c JOIN professor p
      ON p.id = c.taughtby
      JOIN student s
      ON c.year = s.year
WHERE c.ects > 2 AND s.year > 2
GROUP BY p.name, c.ects
```



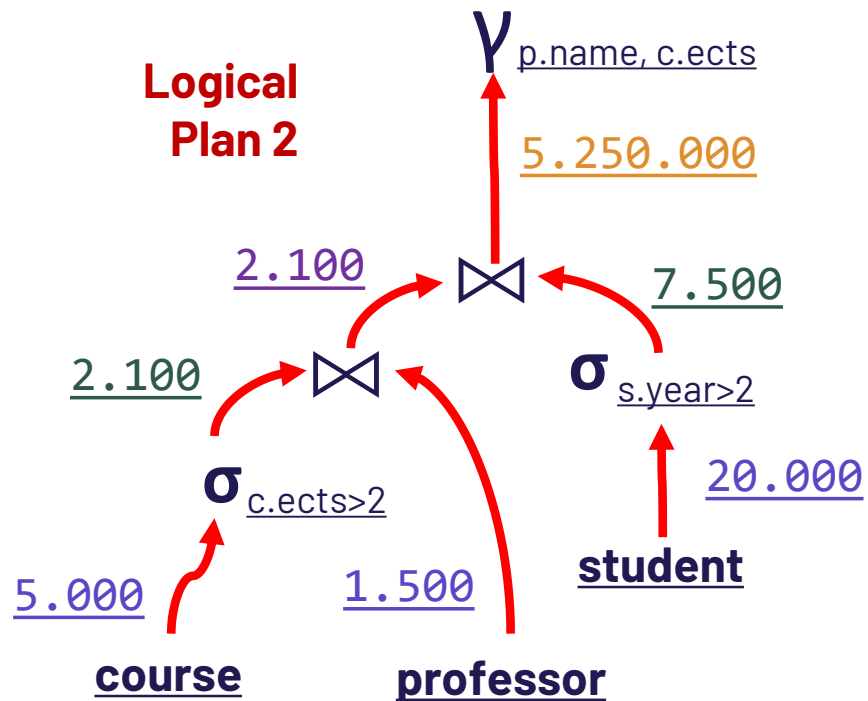
Cost: 5.250.000 + 5.250.000 + 7.500 + 2.100 + 5.000 + 20.000 + 1.500 = 10.536.100

Intermediate Results (alternative)

The cost of the plan is correlated with the amount of data to process = # of tuples

```
SELECT p.name, c.ects, COUNT(*)
FROM course c JOIN professor p
      ON p.id = c.taughtby
      JOIN student s
      ON c.year = s.year
WHERE c.ects > 2 AND s.year > 2
GROUP BY p.name, c.ects
```

How can we make sure we pick a plan that is not too expensive?



Cost: 5.250.000 + 2.100 + 7.500 + 2.100 + 5.000 + 1.500 + 20.000 = 5.288.200



Logical Plan Optimization

Transform a logical plan into an equivalent logical plan using pattern matching rules.

Follow some heuristics rules that are likely to produce a better plan:

- We move from one plan to another equivalent plan (we guarantee the same correct output)
- We use a set of rules:
 - Split Conjunctive Predicates
 - Predicate Pushdown
 - Replace Cartesian Product with Joins
 - Projection Pushdown

We cannot be sure the final execution cost will be better all the time, but usually it works

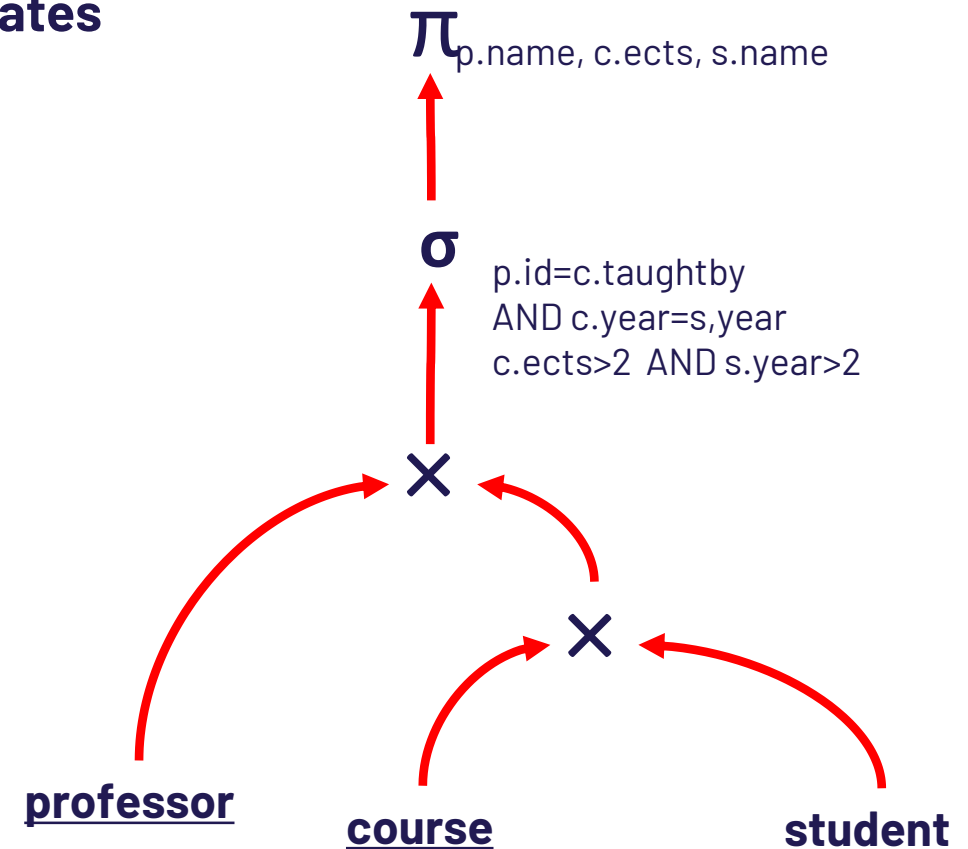




Split Conjunctive Predicates

Decompose a complex predicate in simpler predicates

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```

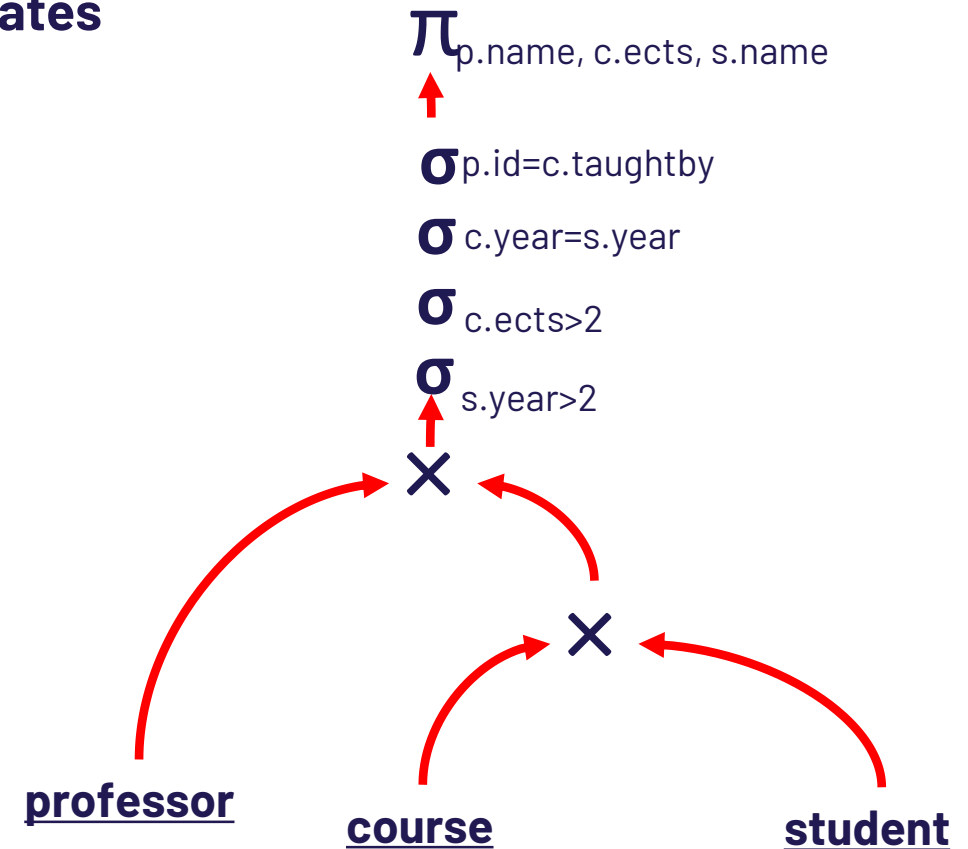




Split Conjunctive Predicates

Decompose a complex predicate in simpler predicates

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```

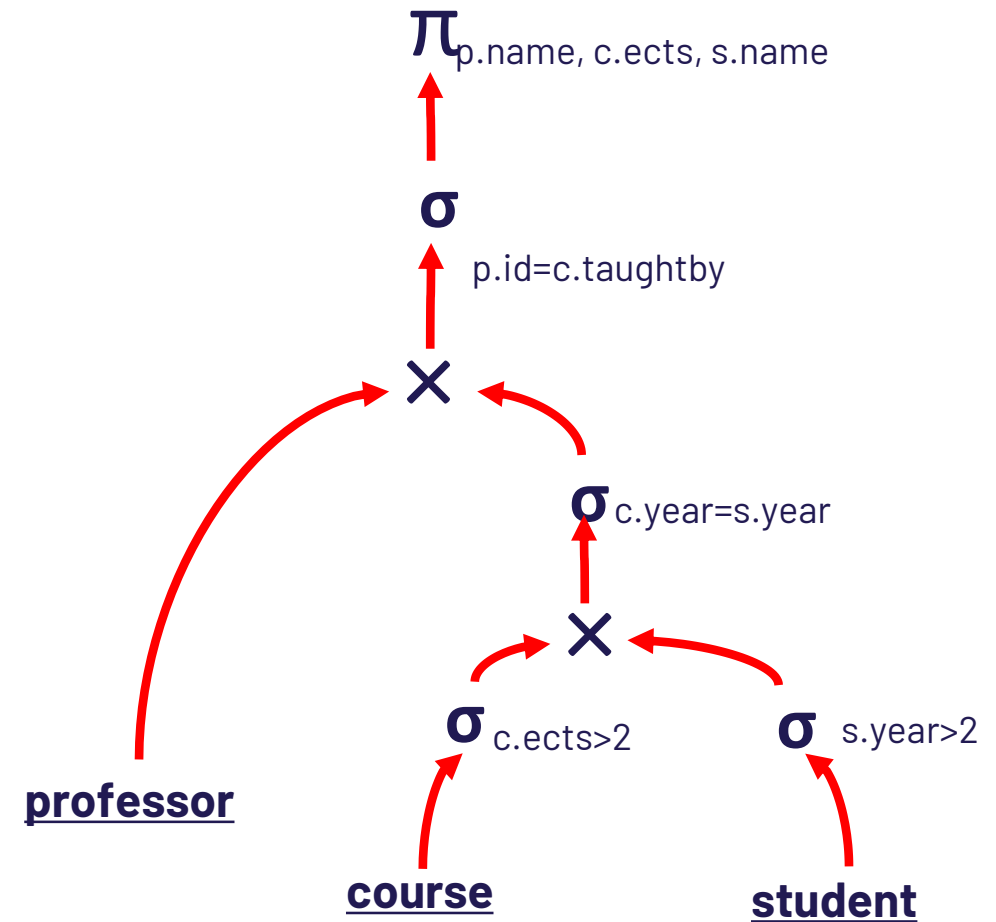




Predicate Pushdown

Move the predicate to the lowest applicable point in the plan

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```

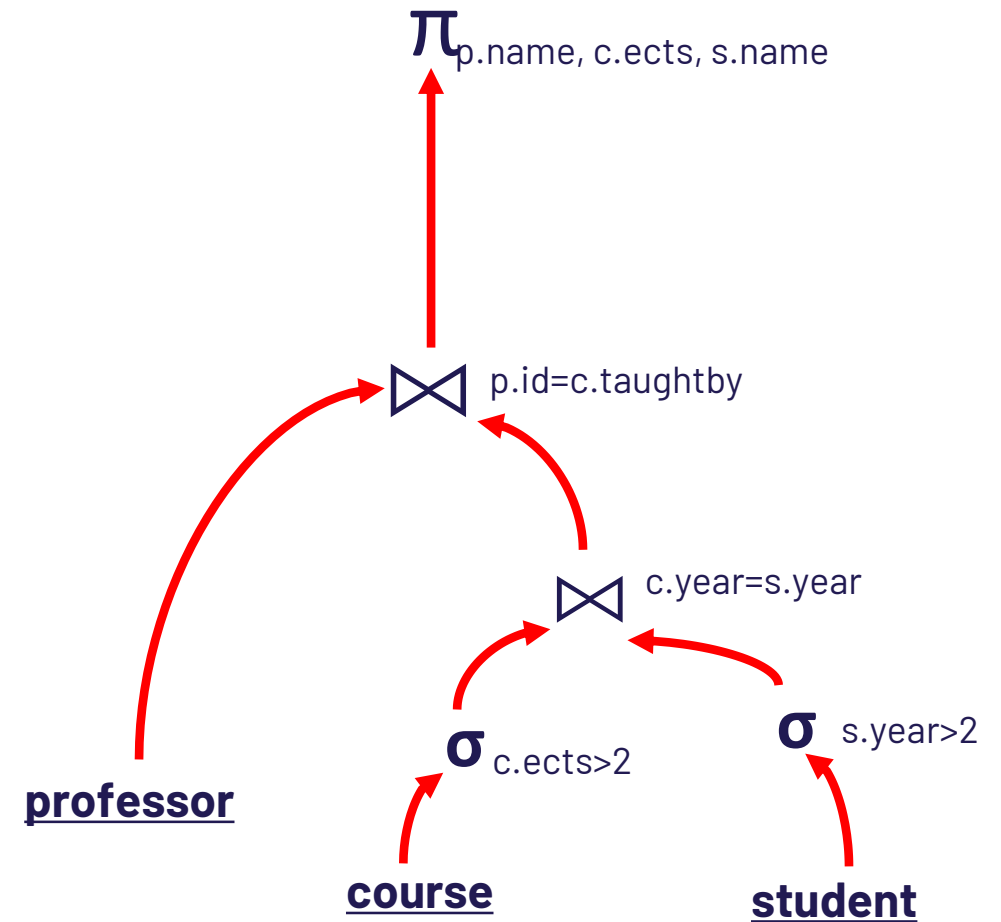




Replace Cartesian Products

Replace all cartesian products with inner joins

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```



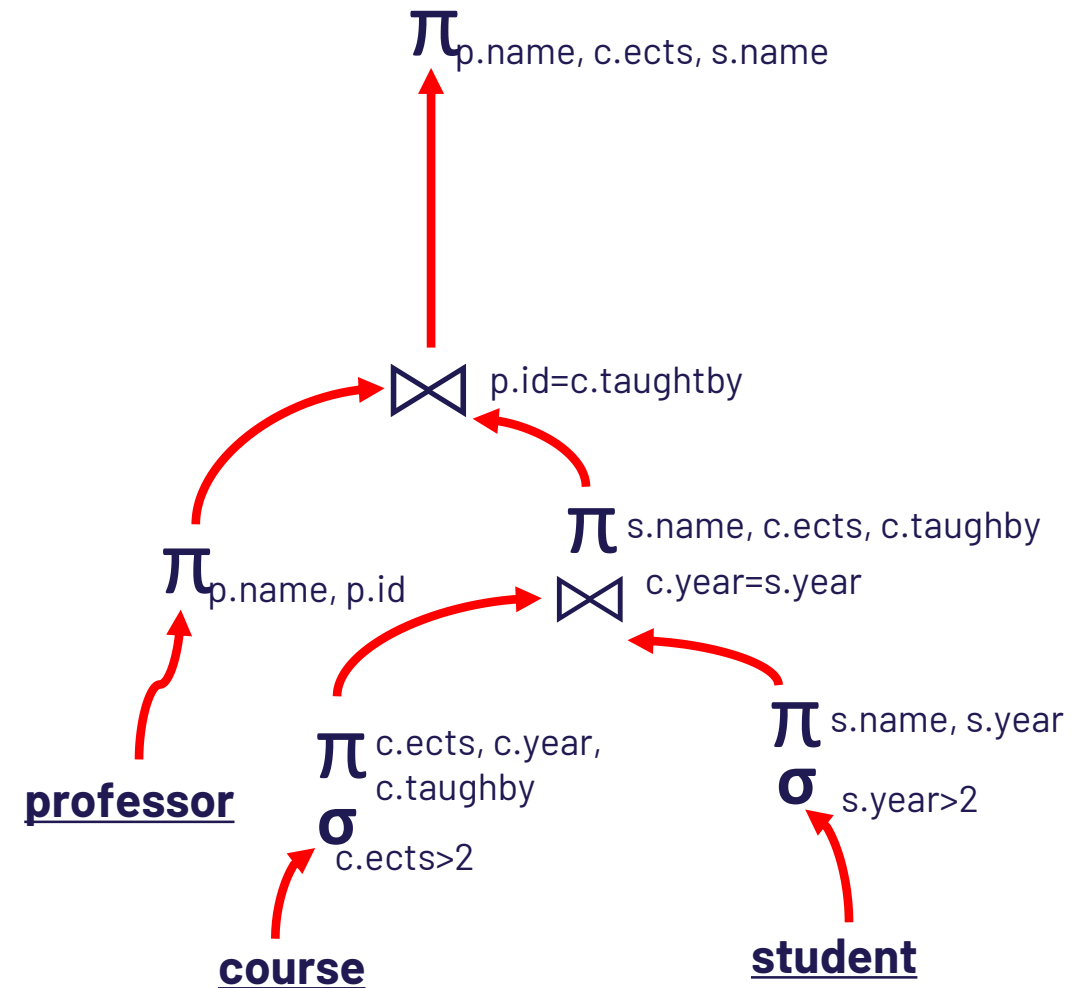


Projection Pushdown

Eliminate redundant attributes before pipeline breakers to reduce materialization cost

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```

Not always useful!



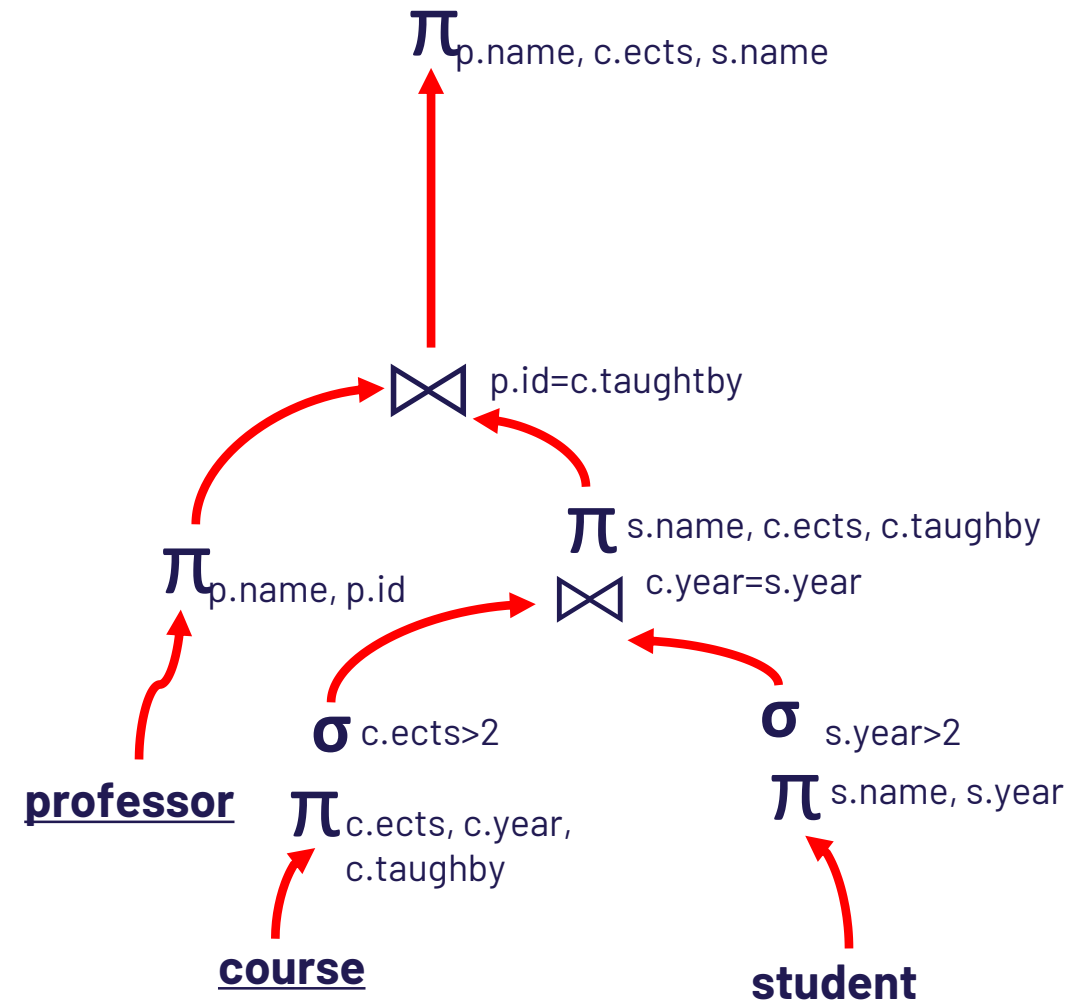


Projection Pushdown (II)

Eliminate redundant attributes before pipeline breakers to reduce materialization cost

```
SELECT p.name, c.ects, s.name
FROM course c, professor p, student s
WHERE p.id = c.taughtby
      AND c.year = s.year
      AND c.ects > 2
      AND s.year > 2
```

Not always useful!



Take away

- ▶ SQL does not determine how the query will be executed.
- ▶ DBMS tries to find a good plan, but it is not optimal.
- ▶ When doing something has an impact on its cost.
- ▶ More on cost estimation on:
 - [PostgreSQL: Documentation: 18: 14.1. Using EXPLAIN](#)
 - [PostgreSQL: Documentation: 18: 14.2. Statistics Used by the Planner](#)